

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO GIỮA KỲ
NHẬP MÔN XỬ LÝ NGÔN NGỮ TỰ NHIÊN**

XÂY DỰNG MÔ HÌNH PHÂN LOẠI CẢM XÚC BÌNH LUẬN TRÊN YOUTUBE

Người hướng dẫn: **T.S Lê Anh Cường**

Người thực hiện: **Nguyễn Mỹ Duyên – 52200190**

Nguyễn Minh Khánh – 52200187

Nguyễn Minh Khang – 52100040

Khoá : 25-26

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO GIỮA KỲ
NHẬP MÔN XỬ LÝ NGÔN NGỮ TỰ NHIÊN**

XÂY DỰNG MÔ HÌNH PHÂN LOẠI CẢM XÚC BÌNH LUẬN TRÊN YOUTUBE

Người hướng dẫn: **T.S Lê Anh Cường**

Người thực hiện: **Nguyễn Mỹ Duyên – 52200190**

Nguyễn Minh Khánh – 52200187

Nguyễn Minh Khang – 52100040

Khoá : 25-26

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025

LỜI CẢM ƠN

Lời nói đầu tiên, chúng em muốn bày tỏ lòng biết ơn đến **Trường Đại học Tôn Đức Thắng** vì đã cung cấp cho chúng em môi trường học tập với cơ sở vật chất hiện đại, một thư viện rộng lớn nhờ đó mà chúng em có thể dễ dàng tìm kiếm, nghiên cứu và chọn lọc các tài liệu quan trọng để hoàn thành bài báo cáo của mình.

Ngoài ra, chúng em cũng muốn gửi lời cảm ơn đến **GV. Lê Anh Cường** vì sự hỗ trợ và chỉ dẫn nhiệt tình trong suốt quá trình này cùng với môi trường học tập mà thầy tạo ra làm cho quá trình học tập và nghiên cứu trở nên thoải mái, linh hoạt hơn, giúp chúng em có thêm động lực để phấn đấu, rèn luyện, và có thêm nhiều ý tưởng giúp ích cho bài báo cáo của mình.

BÁO CÁO ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Chúng tôi xin cam đoan đây là công trình nghiên cứu của riêng chúng tôi và được sự hướng dẫn khoa học của **T.S Lê Anh Cường**. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong báo cáo còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào chúng tôi xin hoàn toàn chịu trách nhiệm về nội dung báo cáo của mình. Trường đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 22 tháng 3 năm 2025

Tác giả

(ký tên và ghi rõ họ tên)



Nguyễn Minh Khánh



Nguyễn Mỹ Duyên

Khang

Nguyễn Minh Khang

PHẦN XÁC NHẬN VÀ ĐÁNH GIÁ CỦA GIẢNG VIÊN

Phần xác nhận của GV hướng dẫn

Tp. Hồ Chí Minh, ngày tháng năm
(kí và ghi họ tên)

Phần đánh giá của GV chấm bài

Tp. Hồ Chí Minh, ngày tháng năm
(kí và ghi họ tên)

TÓM TẮT

Bài báo cáo này trình bày quá trình xây dựng mô hình phân loại cảm xúc bình luận trên youtube bằng các phương pháp học máy. Bao gồm:

- Chương I: Xây dựng tập dữ liệu cho bài toán
- Chương II: Biểu diễn văn bản
- Chương III: Huấn luyện mô hình học máy
- Chương IV: Đánh giá và phân tích kết quả

Những phát hiện và kết quả từ nghiên cứu giúp xác định phương pháp tối ưu để phân loại cảm xúc bình luận trên YouTube, hỗ trợ tự động hóa quy trình phân tích phản hồi từ người dùng.

MỤC LỤC

LỜI CẢM ƠN	i
TÓM TẮT	v
MỤC LỤC	1
DANH MỤC HÌNH ẢNH, BẢNG BIỂU	4
CHƯƠNG 1 – XÂY DỰNG TẬP DỮ LIỆU CHO BÀI TOÁN	6
1.1. Giới thiệu bối cảnh bài toán	6
1.2. Mục tiêu nghiên cứu	6
1.3. Tổng quan về tập dữ liệu	7
1.3.1. Mô tả tập dữ liệu	7
1.3.2. Phương pháp xây dựng tập dữ liệu	7
1.4. Đặc điểm của tập dữ liệu	8
1.5. Tiền xử lý dữ liệu	8
1.6. Gán nhãn cảm xúc cho từng bình luận	9
1.7. Ứng dụng Selenium và Gemini API trong Xây dựng Tập Dữ liệu	9
1.7.1. Cào dữ liệu bình luận từ YouTube trong file <i>Get Youtube Comments and Labeling.ipynb</i>	9
1.7.2. Gán nhãn cảm xúc cho bình luận bằng API Gemini	10
CHƯƠNG 2 – BIỂU DIỄN VĂN BẢN	13
2.1. Giới thiệu về mô hình Bag-of-Words (BoW)	13
2.1.1. Khái niệm	13
2.1.2. Cách hoạt động	14
2.1.3. Ví dụ minh họa	14
2.1.4. Ưu và nhược điểm:	15
2.2. Giới thiệu về mô hình TF-IDF (Term Frequency - Inverse Document Frequency)	16
2.2.1. Khái niệm	16
2.2.2. Nguyên lý hoạt động	16
2.2.3. Công thức	16
2.2.4. Ví dụ minh họa	17
2.2.4. Ưu và nhược điểm	18

2.3.	Giới thiệu về mô hình Binary (Biểu diễn nhị phân)	18
2.3.1.	<i>Khái niệm</i>	18
2.3.2.	<i>Nguyên lý hoạt động</i>	19
2.3.3.	<i>Công thức</i>	19
2.3.4.	<i>Ví dụ minh họa</i>	19
2.3.5.	<i>Ưu và nhược điểm:</i>	20
2.4.	Giới thiệu về mô hình Count Vectorizer (Bag of Words - BoW)	21
2.4.1.	<i>Khái niệm</i>	21
2.4.2.	<i>Nguyên lý hoạt động</i>	21
2.4.3.	<i>Công thức</i>	21
2.4.4.	<i>Ưu và nhược điểm:</i>	22
2.5.	Giới thiệu về mô hình N-gram	22
2.5.1.	<i>Khái niệm</i>	22
2.5.2.	<i>Nguyên lý hoạt động</i>	23
2.5.3.	<i>Công thức</i>	23
2.5.4.	<i>Ưu và nhược điểm:</i>	23
2.6.	Giới thiệu về mô hình Hashing Vectorizer	24
2.6.1.	<i>Khái niệm</i>	24
2.6.2.	<i>Nguyên lý hoạt động</i>	24
2.6.3.	<i>Ưu và nhược điểm:</i>	25
2.7.	Áp dụng các phương pháp học máy truyền thống trên biểu diễn BoW	26
2.7.1.	<i>Thuật toán Naive Bayes (Naive Bayesian Classification)</i>	26
2.7.2.	<i>Thuật toán Logistic Regression trong phân loại văn bản</i>	29
2.7.3.	<i>Thuật toán Decision Tree trong phân loại văn bản</i>	31
2.7.4.	<i>Thuật toán Random Forest</i>	34
2.7.5.	<i>Thuật toán KNN (K-Nearest Neighbors)</i>	38
2.8.	Áp dụng các phương pháp học máy truyền thống bài toán phân loại cảm xúc bình luận trên YouTube	42
2.8.1.	<i>Mục tiêu của bài toán:</i>	42
2.8.2.	<i>Nội dung các bước thực hiện chi tiết</i>	43
CHƯƠNG 3 – HUẤN LUYỆN MÔ HÌNH HỌC MÁY		52
3.1.	Giới thiệu về Doc2Vec	52

3.2.	Các phiên bản mô hình Doc2Vec	52
3.3.	Nguyên lý hoạt động	53
3.4.	Cách triển khai Doc2Vec cho bài toán phân loại cảm xúc bình luận	53
3.4.1	<i>Không sử dụng mô hình pretrain (huấn luyện từ đầu)</i>	53
3.4.2	<i>Sử dụng mô hình pretrain (có sẵn)</i>	54
3.5.	Đánh giá mô hình Doc2Vec và những lưu ý khi huấn luyện	54
3.6.	Ví dụ minh họa	55
3.7.	Giới thiệu về BERT	56
3.7.1.	<i>Mô hình BERT</i>	56
3.7.2.	<i>Các phiên bản mô hình BERT</i>	56
3.7.3.	<i>Nguyên lý hoạt động của BERT</i>	57
3.8.	Cách triển khai BERT cho bài toán phân loại cảm xúc bình luận	57
3.8.1.	<i>Không sử dụng mô hình pretrain (huấn luyện từ đầu)</i>	57
3.8.2.	<i>Sử dụng mô hình pretrain (PhoBERT)</i>	58
3.9.	Hiện thực trên bài toán phân loại cảm xúc bình luận trên youtube	58
3.9.1.	<i>Mục tiêu của bài toán:</i>	58
3.9.2.	<i>Nội dung các bước thực hiện chi tiết</i>	59
3.10.	So sánh giữa Doc2Vec và BERT (from scratch)	69
CHƯƠNG 4 – ĐÁNH GIÁ VÀ PHÂN TÍCH KẾT QUẢ		71
4.1.	So sánh các phương pháp biểu diễn văn bản	71
4.2.	Tổng kết và lựa chọn giải pháp hiệu quả	75
4.3.	Kết luận lựa chọn giải pháp phù hợp cho đề tài	76
4.4.	Đề xuất giải pháp và hướng phát triển cho bài toán "Phân loại cảm xúc bình luận trên youtube"	76
4.4.1.	<i>Giai đoạn 1 (Dữ liệu ít, triển khai nhanh):</i>	76
4.4.2.	<i>Giai đoạn 2 (Dữ liệu mở rộng, ngành nghề đa dạng):</i>	76
4.4.3.	<i>Hướng phát triển:</i>	76
TÀI LIỆU THAM KHẢO		77

DANH MỤC HÌNH ẢNH, BẢNG BIỂU

DANH MỤC HÌNH

Chương 1:

Hình 1. 1 Mô tả tập dữ liệu	7
-----------------------------------	---

Chương 2:

Hình 2. 1 Kết quả biểu đồ cột Binary	46
Hình 2. 2 Kết quả Confusion Matrix Binary	46
Hình 2. 3 Kết quả biểu đồ cột Counting	47
Hình 2. 4 Kết quả Confusion Matrix Counting	47
Hình 2. 5 Kết quả biểu đồ cột N-gram	48
Hình 2. 6 Kết quả Confusion Matrix N-gram	48
Hình 2. 7 Kết quả biểu đồ cột TF-IDF	49
Hình 2. 8 Kết quả Confusion Matrix TF-IDF	49

Chương 3:

Hình 3. 1 Mô hình Doc2Vec	52
Hình 3. 2 Mô hình Bert	56

DANH MỤC BẢNG

Chương 2:

<i>Bảng 2. 1 Biểu diễn mỗi câu dưới dạng vector BoW (dạng One-hot)</i>	15
<i>Bảng 2. 2 Biểu diễn mỗi câu dưới dạng vector BoW (dạng TF-IDF)</i>	17
<i>Bảng 2. 3 Biểu diễn mỗi câu dưới dạng vector Binary</i>	20
<i>Bảng 2. 4 Mô hình phân lớp NB</i>	26
<i>Bảng 2. 5 Mô hình hồi quy Logistic</i>	29
<i>Bảng 2. 6 Mô hình phân lớp DT</i>	31
<i>Bảng 2. 7 Thuật toán Random Forest</i>	34
<i>Bảng 2. 8 KNN (K-Nearest Neighbors)</i>	38

Chương 3:

Bảng 3. 1 So sánh giữa Doc2Vec và BERT	70
--	----

Chương 4:

Bảng 4. 1 So sánh các phương pháp biểu diễn văn bản.....	72
Bảng 4. 2 So sánh về tính đầy đủ và chính xác của dữ liệu.....	73
Bảng 4. 3 So sánh các phương pháp học máy đã áp dụng.....	74
Bảng 4. 4 Lựa chọn giải pháp phù hợp cho đề tài	76

CHƯƠNG 1 – XÂY DỰNG TẬP DỮ LIỆU CHO BÀI TOÁN

1.1. Giới thiệu bối cảnh bài toán

- Trong thời đại công nghệ số, YouTube trở thành một trong những nền tảng chia sẻ video lớn nhất, thu hút hàng triệu bình luận từ người dùng mỗi ngày. Bình luận trên YouTube phản ánh ý kiến, cảm xúc của người xem về nội dung video, từ đó cung cấp dữ liệu quý giá cho các nhà sáng tạo nội dung, doanh nghiệp và cộng đồng.
- Tuy nhiên, việc phân tích và phân loại cảm xúc trong bình luận là một thách thức lớn do khối lượng dữ liệu khổng lồ, sự đa dạng về ngôn ngữ, phong cách diễn đạt và ngữ cảnh. Xây dựng mô hình phân loại cảm xúc giúp hiểu rõ hơn về phản hồi của người xem, hỗ trợ cải thiện chất lượng nội dung và tối ưu hóa chiến lược truyền thông.

1.2. Mục tiêu nghiên cứu

- Nghiên cứu này nhằm xây dựng một hệ thống phân loại cảm xúc bình luận trên YouTube bằng cách áp dụng các phương pháp học máy. Thực hiện với các bước:
 - Thu thập dữ liệu thực tế bằng cách cào bình luận trên YouTube và gán nhãn dữ liệu bằng cách gọi API của Gemini thay vì sử dụng mô hình LLM để sinh dữ liệu.
 - Áp dụng các thuật toán học máy khác nhau để phân loại bình luận theo các cảm xúc tích cực, tiêu cực hoặc trung lập.
 - Đánh giá và so sánh hiệu suất của các phương pháp phân loại.

1.3. Tổng quan về tập dữ liệu

1.3.1. Mô tả tập dữ liệu

	A	B
1	Comment_raw	sentiment
2	đưa nào nói dối tao	Neg
3	Top các bài hát bị n -Cái bài hát thật về l	Neg
4	Ai từ DXY đây?	Neu
5	Đây là thảm họa của	Neg
6	top những bài hát bị	Neu
7	Ngày xưa bài này đư Nhưng sau mấy năm	Neu
8	Tính ra đoạn điệp kh	Neg
9	top MV bị ném đá đi	Neg
10	Nghe cũng hay phết	Pos

Hình 1. 1 Mô tả tập dữ liệu

- Tập dữ liệu được sử dụng trong nghiên cứu này bao gồm các bình luận trên YouTube được thu thập và gắn nhãn bằng API của Gemini. Mỗi bình luận chứa các thông tin sau:
- Nội dung bình luận (**Comment_raw**): Văn bản bình luận từ người dùng trên YouTube.
- Cảm xúc (**sentiment**): Nhãn của bình luận, bao gồm ba loại chính:
 - **Pos (Positive)**: Bình luận mang tính tích cực.
 - **Neg (Negative)**: Bình luận mang tính tiêu cực.
 - **Neu (Neutral)**: Bình luận trung lập, không thể hiện rõ cảm xúc tích cực hay tiêu cực.
- Tập dữ liệu này giúp mô hình học máy nhận diện và phân loại chính xác cảm xúc của bình luận, từ đó hỗ trợ phân tích phản hồi của người dùng trên YouTube một cách tự động và hiệu quả.

1.3.2. Phương pháp xây dựng tập dữ liệu

Tập dữ liệu này được xây dựng thông qua hai phương pháp chính:

- Thu thập dữ liệu từ YouTube bằng Web Scraping:

- Sử dụng Selenium để cào dữ liệu bình luận từ các video YouTube.
- Tự động cuộn trang để lấy nhiều bình luận hơn.
- Lọc bỏ các bình luận không hợp lệ (spam, quá ngắn, không liên quan, không phải tiếng Anh hoặc tiếng Việt).
- Lưu dữ liệu vào file CSV để phục vụ quá trình gán nhãn và huấn luyện mô hình.
- Gán nhãn tự động bằng API Gemini:
 - Sử dụng API Gemini để phân tích và gán nhãn cảm xúc cho bình luận.
 - Xác định nhãn cảm xúc dựa trên nội dung bình luận (tích cực, tiêu cực, trung lập).
 - Kiểm tra lại một phần dữ liệu để đảm bảo độ chính xác của nhãn.

1.4. Đặc điểm của tập dữ liệu

- Số lượng mẫu: Hàng nghìn bình luận trên YouTube.
- Số lượng nhãn cảm xúc: Ba nhãn chính gồm Positive (Pos), Negative (Neg), Neutral (Neu).
- Định dạng: Dữ liệu được lưu dưới dạng CSV để dễ dàng xử lý.
- Ngôn ngữ: Tiếng Anh và tiếng Việt.
- Chất lượng dữ liệu:
 - Loại bỏ spam, bình luận không liên quan.
 - Gán nhãn tự động bằng API Gemini và kiểm tra lại để đảm bảo độ chính xác.

1.5. Tiền xử lý dữ liệu

- Loại bỏ dữ liệu trùng lặp: Nếu có nhiều bình luận giống nhau từ các video khác nhau hoặc trùng lặp trong quá trình cào dữ liệu.
- Chuẩn hóa văn bản:
 - Chuyển toàn bộ bình luận về chữ thường.

- Loại bỏ các ký tự đặc biệt không cần thiết (emoji, dấu câu thừa).
- Bỏ dấu tiếng Việt để tránh lỗi encoding.
- Làm sạch dữ liệu:
 - Loại bỏ các bình luận quá ngắn hoặc không mang ý nghĩa rõ ràng.
 - Kiểm tra và sửa lỗi chính tả nếu cần thiết.

1.6. Gán nhãn cảm xúc cho từng bình luận

- Sử dụng danh mục nhãn cảm xúc chuẩn: Positive (Pos), Negative (Neg), Neutral (Neu).
- Nếu dữ liệu đã có nhãn, kiểm tra lại tính chính xác để đảm bảo độ tin cậy.
- Nếu chưa có nhãn, sử dụng API Gemini để tự động phân loại cảm xúc dựa trên nội dung bình luận.

1.7. Ứng dụng Selenium và Gemini API trong Xây dựng Tập Dữ liệu

1.7.1. Cào dữ liệu bình luận từ YouTube trong file *Get Youtube Comments and Labeling.ipynb*

Để thu thập dữ liệu bình luận từ YouTube, nhóm em đã áp dụng phương pháp Web Scraping sử dụng thư viện Selenium – một công cụ cho phép tự động hóa trình duyệt web. Quá trình thực hiện bao gồm các bước sau:

- **Chuẩn bị môi trường và cài đặt thư viện** Selenium, webdriver_manager và langdetect.

```
from selenium import webdriver
from langdetect import detect
```

- **Cấu hình trình duyệt và tự động cuộn trang** để tiến hành thu thập bình luận, nhóm sử dụng cấu hình trình duyệt ở chế độ headless, nghĩa là chạy mà không hiển thị giao diện người dùng, giúp tăng tốc độ và hiệu quả. Các tùy chọn cấu hình như vô hiệu hóa GPU, tắt sandbox và đặt user-agent được thiết lập nhằm tránh các giới hạn từ phía YouTube.

```
chrome_options = Options()
```



```
chrome_options.add_argument("--headless")
chrome_options.add_argument("--disable-gpu")
chrome_options.add_argument("--no-sandbox")
chrome_options.add_argument("user-agent=Mozilla/5.0")
```

– **Truy xuất và lưu trữ dữ liệu :**

- Bình luận Sau khi tải trang và thực hiện cuộn trang, các phần tử chứa bình luận được xác định thông qua bộ chọn CSS.

```
driver.get("https://www.youtube.com/watch?v=example")
comment_elements =
driver.find_elements(By.CSS_SELECTOR, "yt-attributed-
string#content-text")
comment = comment_elements[0].text.strip()
print(f"Bình luận: {comment}")
```

- Nội dung của các bình luận được trích xuất, làm sạch và kiểm tra hợp lệ. Ngoài ra, để đảm bảo dữ liệu thu thập được chỉ chứa các bình luận bằng tiếng Anh và tiếng Việt, nhóm sử dụng thư viện langdetect nhằm lọc bỏ những bình luận không đạt yêu cầu ngôn ngữ.

```
lang in ["en", "vi"]
```

- **Thu thập dữ liệu:** trình duyệt tự động cuộn trang và thu thập tất cả các bình luận hoặc có thể giới hạn số bình luận có sẵn cho đến khi không còn bình luận mới được load. Dữ liệu thu thập được lưu trữ vào file CSV với định dạng mã hóa UTF-8 để đảm bảo hiển thị chính xác tiếng Việt. Các trường dữ liệu được lưu lại bao gồm nội dung bình luận (Comment_raw).

1.7.2. Gán nhãn cảm xúc cho bình luận bằng API Gemini

Sau khi thu thập dữ liệu bình luận từ YouTube, bước tiếp theo là gán nhãn cảm xúc cho mỗi bình luận, xác định xem bình luận đó mang tính tích cực (Positive), tiêu cực (Negative) hay trung lập (Neutral). Để tự động hóa quá trình phân loại

cảm xúc, nhóm em đã sử dụng API Gemini của Google – một công cụ xử lý ngôn ngữ tự nhiên:

- **Cài đặt và cấu hình:** thư viện google-generativeai để tương tác với API Gemini và pandas để xử lý dữ liệu dạng bảng. API được cấu hình thông qua việc thiết lập API key của Google, cho phép xác thực các yêu cầu gửi đến dịch vụ Gemini

```
import google.generativeai as genai
GOOGLE_API_KEY = "my-key"
genai.configure(api_key=GOOGLE_API_KEY)
```

- **Xây dựng hàm phân loại cảm xúc**
 - Quá trình phân loại cảm xúc được thực hiện thông qua một hàm chuyên dụng. Trong hàm này, mỗi bình luận được đưa vào một prompt (lời nhắc) được xây dựng dưới dạng văn bản mô tả yêu cầu phân loại, ví dụ: “Phân loại cảm xúc của bình luận sau thành Positive, Negative hoặc Neutral ...”. Prompt này được gửi đến mô hình Gemini, yêu cầu trả về một từ duy nhất (Pos, Neg hoặc Neu) tương ứng với nhãn cảm xúc.

```
comment = "Video này rất hay!"
prompt = f"Phân loại cảm xúc: '{comment}' -> Pos, Neg hoặc Neu"
sentiment = classify_comment(comment)
print(f"Cảm xúc: {sentiment}") # Kết quả: Pos
```

- Để tăng tính ổn định của quá trình gọi API, hàm phân loại được xây dựng với cơ chế retry – tức là nếu có lỗi xảy ra trong quá trình gửi yêu cầu (do kết nối hay giới hạn tần suất), hàm sẽ tự động thử lại theo cơ chế chậm dần (exponential backoff). Khi vượt quá số lần thử quy định, hàm sẽ mặc định trả về nhãn "Neutral".
- **Xử lý dữ liệu và ghi kết quả**

- Quá trình gán nhãn được thực hiện trên từng bình luận trong file dữ liệu được tạo nên từ việc cào dữ liệu ở bước trước. Mỗi bình luận sau khi được xử lý qua hàm phân loại sẽ được ghi lại vào một file CSV mới và kèm theo cột ghi nhãn cảm xúc (Sentiment).
- Quy trình này bao gồm:
 - Đọc dữ liệu: Tải dữ liệu bình luận từ file CSV.
 - Duyệt qua từng bình luận: Với mỗi bình luận, hàm phân loại được gọi để xác định nhãn cảm xúc dựa trên nội dung văn bản.
 - Ghi kết quả: Kết quả được ghi vào file CSV mới gồm 2 cột của bình luận (Comment_raw) và cột cảm xúc(Sentiment).
- **Ưu điểm và hạn chế:**
 - Ưu điểm:
 - Tự động hóa việc gán nhãn cảm xúc cho số lượng lớn bình luận, giảm thiểu thời gian và công sức so với xử lý thủ công.
 - Nhãn cảm xúc được gán dựa trên một mô hình tiên tiến, cho kết quả nhất quán trên nhiều bình luận khác nhau.
 - Quy trình có thể được áp dụng cho số lượng bình luận lớn, với khả năng xử lý linh hoạt giữa thu thập không giới hạn và thu thập có giới hạn.
 - Hạn chế:
 - Quá trình phân loại phụ thuộc vào API Gemini, do đó yêu cầu phải có kết nối Internet ổn định và khả năng xử lý theo giới hạn tần suất của API.
 - Mặc dù có cơ chế retry, nhưng vẫn có khả năng một số bình luận không được phân loại chính xác do lỗi hệ thống hoặc các yếu tố khác.

- **Gắn nhãn thủ công:** để tránh tình trạng retry và gắn nhãn sai sót của API, nhóm đã theo dõi quá trình gắn nhãn, kiểm tra và chỉnh sửa nhãn cho phù hợp với từng bình luận

CHƯƠNG 2 – BIỂU DIỄN VĂN BẢN

2.1. Giới thiệu về mô hình Bag-of-Words (BoW)

2.1.1. Khái niệm

- Mô hình Bag-of-Words (BoW) là một trong những phương pháp đơn giản nhưng rất hiệu quả để biểu diễn văn bản trong các bài toán xử lý ngôn ngữ tự nhiên (Natural Language Processing - NLP). BoW được sử dụng phổ biến nhằm mục đích chuyển đổi các câu chữ, đoạn văn bản thành dạng số liệu có thể xử lý được bằng các thuật toán học máy.
- Ý tưởng cốt lõi của mô hình Bag-of-Words là xem mỗi câu, đoạn văn bản như một “túi từ” (bag of words), trong đó chỉ tập trung vào việc các từ gì xuất hiện và tần suất xuất hiện của chúng là bao nhiêu, mà không quan tâm đến vị trí, ngữ pháp hay ngữ cảnh xung quanh từ đó. Điều này có nghĩa là dù các từ có thể sắp xếp lại vị trí, nhưng khi xuất hiện giống nhau và số lần xuất hiện không thay đổi thì biểu diễn BoW của câu vẫn giữ nguyên.
- Về mặt kỹ thuật, BoW tiến hành biểu diễn mỗi câu hoặc văn bản dưới dạng một vector có số chiều bằng số lượng từ vựng (từ duy nhất) trong tập dữ liệu. Mỗi chiều trong vector này tương ứng với một từ cụ thể, và giá trị của nó là số lần từ đó xuất hiện trong văn bản. Ví dụ, nếu từ "học" xuất hiện 3 lần trong câu thì ở vị trí tương ứng với từ "học" trong vector sẽ mang giá trị 3.
- Có thể coi cách tiếp cận này như một dạng học biểu diễn đơn giản, trong đó mỗi câu được ánh xạ vào không gian N chiều (với N là tổng số từ vựng). Trong không gian này, mỗi câu được gán một vector duy nhất, thể hiện đặc điểm riêng biệt của câu đối với mô hình.

- Tuy BoW bỏ qua hoàn toàn yếu tố ngữ nghĩa, ngữ cảnh và cấu trúc ngữ pháp của câu, nhưng sức mạnh của mô hình lại nằm ở sự đơn giản và hiệu quả khi áp dụng cho các tác vụ như phân loại văn bản (text classification), phát hiện spam, phân tích cảm xúc (sentiment analysis), và nhiều bài toán NLP khác.
- Nhìn chung, Bag-of-Words là phương pháp dễ triển khai, dễ hiểu, phù hợp với những bài toán cần biểu diễn nhanh chóng và đơn giản hóa dữ liệu văn bản để phục vụ cho các mô hình học máy truyền thống như Naive Bayes, Logistic Regression, hoặc Decision Tree.

2.1.2. Cách hoạt động

BoW hoạt động theo các bước chính sau:

- Thu thập tập dữ liệu văn bản: Tập hợp các văn bản cần xử lý.
- Tiền xử lý văn bản:
 - Chuyển đổi chữ hoa thành chữ thường.
 - Loại bỏ dấu câu, ký tự đặc biệt.
 - Loại bỏ stopwords (các từ phổ biến như "và", "của", "là").
 - Stemming hoặc Lemmatization (chuẩn hóa từ về dạng gốc).
- Xây dựng từ điển (Vocabulary): Tạo danh sách tất cả các từ duy nhất trong tập dữ liệu.
- Mã hóa văn bản: Biểu diễn mỗi văn bản dưới dạng một vector, trong đó mỗi văn bản sẽ được biểu diễn thành một vector với:
 - Mỗi chiều (cột) tương ứng với một từ trong từ vựng.
 - Giá trị ở mỗi chiều là số lần từ đó xuất hiện trong văn bản.

2.1.3. Ví dụ minh họa

- Giả sử tập dữ liệu gồm 3 bình luận đơn giản sau:
 - Bình luận 1: "Video rất hay!"
 - Bình luận 2: "Không thích nội dung."
 - Bình luận 3: "Chất lượng ổn."

*** Bước 1:** Xây dựng từ điển

Từ điển (Vocabulary) gồm các từ duy nhất:

["video", "rất", "hay", "không", "thích", "nội", "dung", "chất", "lượng", "ổn"]

*** Bước 2:** Biểu diễn mỗi câu dưới dạng vector BoW (dạng One-hot)

Từ	video	rất	hay	không	thích	nội	dung	chất	lượng	ổn
Bình luận 1	1	1	1	0	0	0	0	0	0	0
Bình luận 2	0	0	0	1	1	1	1	0	0	0
Bình luận 3	0	0	0	0	0	0	0	1	1	1

Bảng 2. 1 Biểu diễn mỗi câu dưới dạng vector BoW (dạng One-hot)

Như vậy, mỗi bình luận được biểu diễn thành vector số học, phục vụ cho mô hình phân loại cảm xúc như tích cực, tiêu cực, trung lập.

2.1.4. Ưu và nhược điểm:

- Ưu điểm:
 - Dễ dàng triển khai và tính toán nhanh trên tập dữ liệu lớn.
 - Phù hợp với bài toán phân loại cảm xúc có số lượng từ vựng hạn chế.
- Nhược điểm:
 - Không nhận diện được sắc thái cảm xúc tinh tế trong bình luận (ví dụ: "Không tệ lắm" có nghĩa tích cực nhưng BoW có thể hiểu sai).
 - Không tận dụng được ngữ cảnh hoặc cách diễn đạt gián tiếp.

2.2. Giới thiệu về mô hình TF-IDF (Term Frequency - Inverse Document Frequency)

2.2.1. Khái niệm

- TF-IDF là một phương pháp biểu diễn văn bản dưới dạng số học, giúp đánh giá mức độ quan trọng của một từ trong một bình luận cụ thể so với toàn bộ tập bình luận. Trong bài toán phân loại cảm xúc trên YouTube, TF-IDF giúp xác định những từ có ảnh hưởng lớn đến việc phân loại cảm xúc, chẳng hạn như "tuyệt vời", "chán", "không thích", giúp mô hình nhận diện cảm xúc chính xác hơn.

2.2.2. Nguyên lý hoạt động

- Tiền xử lý và xây dựng từ điển như các phương pháp trên.
- Tính TF (tần suất từ) của từng từ trong từng văn bản.
- Tính IDF (độ hiếm của từ) trên toàn bộ tập văn bản.
- Nhân TF và IDF để thu được trọng số TF-IDF cho từng từ.
- Vector TF-IDF phản ánh mức độ quan trọng của từ trong từng văn bản.

2.2.3. Công thức

- TF (Term Frequency - Tần suất xuất hiện của từ):
 - Đo lường số lần một từ xuất hiện trong một văn bản, so với tổng số từ trong văn bản đó.
 - Công thức:

$$TF(t, d) = \frac{\text{Số lần từ } t \text{ xuất hiện trong văn bản } d}{\text{Tổng số từ trong văn bản } d}$$

- IDF (Inverse Document Frequency - Tần suất nghịch đảo của văn bản):
 - Đo lường tầm quan trọng của một từ, bằng cách giảm trọng số của các từ xuất hiện ở nhiều văn bản (vì chúng ít mang tính phân biệt).
 - Công thức:

$$IDF(t) = \log \left(\frac{N}{df(t)} \right)$$

Với:

- N : Tổng số văn bản trong tập dữ liệu.
- $df(t)$: Số văn bản chứa từ t .

- Được tính bằng tích của TF và IDF

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

2.2.4. Ví dụ minh họa

– Giả sử tập dữ liệu gồm 3 bình luận sau:

- Bình luận 1: "Video rất hay."
- Bình luận 2: "Không thích nội dung."
- Bình luận 3: "Âm thanh tốt."

* **Bước 1:** Xây dựng từ điển

Từ điển (Vocabulary) gồm các từ duy nhất: ["video", "hay", "thích", "tốt"]

* **Bước 2:** Biểu diễn mỗi câu dưới dạng vector BoW (dạng TF-IDF)

Từ	Bình luận 1(TF-IDF)	Bình luận 2(TF-IDF)	Bình luận 3(TF-IDF)
hay	Cao	0	0
thích	0	Cao	0
tốt	0	0	Cao

Bảng 2. 2 Biểu diễn mỗi câu dưới dạng vector BoW (dạng TF-IDF)

Như vậy, từ "hay" có trọng số cao trong Bình luận 1, giúp nhận diện cảm xúc tích cực. Từ "thích" xuất hiện trong Bình luận 2 giúp phân loại cảm xúc tiêu cực hoặc trung lập, trong khi từ "tốt" trong Bình luận 3 thể hiện cảm xúc tích cực.

2.2.4. Ưu và nhược điểm

- Ưu điểm:
 - Phản ánh rõ tầm quan trọng của từ: Kết hợp giữa tần suất và mức độ phân biệt, giúp mô hình chú ý hơn đến các từ đặc trưng.
 - Giảm trọng số các từ phổ biến: Tránh việc các từ như “là”, “có”, “và” chiếm ưu thế trong biểu diễn.
 - Hiệu quả cao trong các bài toán phân loại: Đặc biệt khi kết hợp với các thuật toán học máy như SVM, Naive Bayes.
- Nhược điểm:
 - Phức tạp: Cần tính toán tần suất và logarit, mất thêm thời gian xử lý dữ liệu.
 - Không xử lý được ngữ cảnh: Dù tính được độ quan trọng, TF-IDF vẫn là kỹ thuật "bag of words", không xét thứ tự từ hay cấu trúc câu.
 - Có thể kém hiệu quả với văn bản cực ngắn: Vì dữ liệu quá ngắn không đủ để tính tần suất và IDF chính xác.

2.3. Giới thiệu về mô hình Binary (Biểu diễn nhị phân)

2.3.1. Khái niệm

- Binary là một phương pháp biểu diễn văn bản dưới dạng vector nhị phân (0 hoặc 1), nhằm xác định sự có mặt hay không có mặt của từ trong một văn bản. Khác với TF-IDF – đánh giá tần suất và mức độ quan trọng – Binary chỉ quan tâm đến sự xuất hiện, nên thường đơn giản và ít bị ảnh hưởng bởi độ dài văn bản.
- Trong bài toán phân loại cảm xúc trên YouTube, mô hình Binary giúp xác định sự có mặt của các từ đặc trưng cảm xúc, ví dụ như “tuyệt vời”, “kinh khủng”, “không thích”, để hỗ trợ mô hình học cách phân biệt các loại cảm xúc khác nhau.

2.3.2. Nguyên lý hoạt động

- Tiền xử lý dữ liệu văn bản: loại bỏ ký tự đặc biệt, chuyển về chữ thường, tách từ.
- Xây dựng từ điển chứa tất cả các từ duy nhất trong tập dữ liệu.
- Mỗi văn bản được biểu diễn bằng một vector nhị phân.
- Nếu từ xuất hiện trong văn bản $\rightarrow$ gán giá trị 1, ngược lại là 0.
- Vector đầu ra phản ánh sự xuất hiện hay không của các từ, không quan tâm số lần.

2.3.3. Công thức

- Với mỗi từ trong từ điển (Vocabulary), gán:
 - 1 nếu từ đó xuất hiện trong văn bản.
 - 0 nếu từ đó không xuất hiện.
- Công thức tổng quát:

$$Binary(t, d) = \begin{cases} 1 & \text{nếu } t \in d \\ 0 & \text{nếu } t \notin d \end{cases}$$

Trong đó:

- t là từ trong từ điển
- d là văn bản (bình luận) cần biểu diễn

2.3.4. Ví dụ minh họa

- Giả sử tập dữ liệu gồm 3 bình luận sau:
 - Bình luận 1: "Video rất hay."
 - Bình luận 2: "Không thích nội dung."
 - Bình luận 3: "Âm thanh tốt."

* **Bước 1:** Xây dựng từ điển

Từ điển (Vocabulary) gồm các từ duy nhất: ["video", "hay", "thích", "tốt"]

*** Bước 2:** Biểu diễn mỗi câu dưới dạng vector Binary

Từ	Bình luận 1(TF-IDF)	Bình luận 2(TF-IDF)	Bình luận 3(TF-IDF)
hay	1	0	0
thích	0	1	0
tốt	0	0	1

Bảng 2. 3 Biểu diễn mỗi câu dưới dạng vector Binary

Như vậy, từ “hay” có mặt trong Bình luận 1 giúp nhận biết cảm xúc tích cực. “Thích” trong Bình luận 2 có thể gợi ý về cảm xúc tiêu cực nếu đi kèm “không”, và từ “tốt” trong Bình luận 3 biểu thị cảm xúc tích cực.

2.3.5. Ưu và nhược điểm:

- Ưu điểm:
 - Đơn giản và hiệu quả: Việc chỉ quan tâm đến sự có mặt của từ giúp giảm độ phức tạp tính toán.
 - Giảm ảnh hưởng của từ lặp lại: Một từ dù xuất hiện nhiều cũng chỉ tính 1 lần, tránh việc từ đó “lấn át” các đặc trưng khác.
 - Phù hợp với văn bản ngắn: Trong các bình luận YouTube – thường là các câu ngắn – Binary có thể phản ánh rõ ràng sự hiện diện của các từ mang cảm xúc.
- Nhược điểm:
 - Thiếu thông tin định lượng: Không phân biệt được tần suất từ, có thể bỏ sót các từ có trọng số cao.
 - Không phản ánh mức độ quan trọng: Một từ phổ biến (như "rất", "lắm") và từ đặc trưng (như "tuyệt vời", "kinh khủng") được coi như nhau.

2.4. Giới thiệu về mô hình mô hình Count Vectorizer (Bag of Words - BoW)

2.4.1. Khái niệm

- Count Vectorizer (hay còn gọi là Bag of Words - BoW) là một phương pháp biểu diễn văn bản dựa trên tần suất xuất hiện của từ trong văn bản, không xét đến vị trí hay ngữ cảnh của từ. Mỗi văn bản được biểu diễn thành một vector, trong đó mỗi phần tử tương ứng với số lần xuất hiện của một từ trong từ điển.
- Trong bài toán phân loại cảm xúc bình luận YouTube, Count Vectorizer giúp mô hình học được các từ có tần suất cao gắn với cảm xúc nhất định, ví dụ như từ “thích” xuất hiện thường xuyên trong các bình luận tích cực.

2.4.2. Nguyên lý hoạt động

- Tiền xử lý văn bản như ở Binary.
- Xây dựng từ điển chứa tất cả các từ duy nhất.
- Mỗi văn bản được biểu diễn bằng một vector có độ dài bằng số từ trong từ điển.
- Mỗi phần tử trong vector chứa số lần từ đó xuất hiện trong văn bản.
- Vector được dùng làm đầu vào cho mô hình phân loại.

2.4.3. Công thức

- Với mỗi từ trong từ điển, giá trị tại vị trí tương ứng trong vector chính là số lần từ đó xuất hiện trong văn bản.
- Công thức tổng quát:

$$Count(t, d) = \text{số lần từ } t \text{ xuất hiện trong văn bản } d$$

Trong đó:

- t : từ trong từ điển
- d : văn bản cần biểu diễn

2.4.4. Ưu và nhược điểm:

- Ưu điểm:
 - Đơn giản, dễ hiểu và triển khai: Count là phương pháp nền tảng, phù hợp cho người mới bắt đầu.
 - Phù hợp với văn bản có độ dài trung bình: Vì thể hiện được tần suất từ, giúp mô hình nhận biết độ mạnh yếu của cảm xúc.
 - Hiệu quả với các mô hình truyền thống như Naive Bayes, Logistic Regression.
- Nhược điểm:
 - Không phản ánh tầm quan trọng toàn cục: Các từ phổ biến vẫn có thể có trọng số cao, làm mô hình hiểu sai ý nghĩa.
 - Không xét ngữ cảnh hay thứ tự từ: Giống như TF-IDF, Count không phân biệt "không thích" với "thích không".
 - Dễ bị ảnh hưởng bởi từ xuất hiện nhiều lần: Các từ lặp lại nhiều sẽ chiếm ưu thế, kể cả khi không thực sự quan trọng.

2.5. Giới thiệu về mô hình N-gram

2.5.1. Khái niệm

- N-gram là một phương pháp biểu diễn văn bản bằng cách xét các chuỗi liên tiếp gồm N từ, thay vì chỉ xét từng từ đơn lẻ. Phương pháp này giúp mô hình học được ngữ cảnh và cấu trúc cơ bản của câu, điều mà các phương pháp như Binary, Count, hay TF-IDF không thể làm được.

- Trong bài toán phân loại cảm xúc bình luận YouTube, N-gram giúp phát hiện những cụm từ mang ý nghĩa cảm xúc rõ rệt, ví dụ như “rất hay”, “không thích”, “quá chán” – những cụm từ mà nếu tách riêng từng từ ra thì rất dễ gây hiểu lầm cho mô hình.

2.5.2. Nguyên lý hoạt động

- Tiền xử lý văn bản và tách từ.
- Thay vì xét từ đơn lẻ, tạo các chuỗi liên tiếp gồm N từ (N-grams).
- Xây dựng từ điển gồm tất cả các N-gram xuất hiện trong tập dữ liệu.
- Biểu diễn mỗi văn bản bằng vector thể hiện số lần xuất hiện của từng N-gram.
- Vector được dùng cho mô hình học để nắm ngữ cảnh từ vựng.

2.5.3. Công thức

- Với mỗi từ trong từ điển, giá trị tại vị trí tương ứng trong vector chính là số lần từ đó xuất hiện trong văn bản.
- Công thức tổng quát:
 - Với văn bản gồm chuỗi các từ:
 - $[w_1, w_2, w_3, \dots, w_n]$
 - N-gram sẽ tạo ra các chuỗi liên tiếp gồm N từ:
 - Uni-gram (N=1): từng từ đơn lẻ
 - Bi-gram (N=2): chuỗi gồm 2 từ liên nhau
 - Tri-gram (N=3): chuỗi gồm 3 từ liên nhau
 - Ví dụ với câu: "Không thích video này"
 - Uni-gram: ["Không", "thích", "video", "này"]
 - Bi-gram: ["Không thích", "thích video", "video này"]
 - Tri-gram: ["Không thích video", "thích video này"]

2.5.4. Ưu và nhược điểm:

- Ưu điểm:

- Giữ được ngữ cảnh cơ bản giữa các từ: Giúp mô hình hiểu được các cụm từ mang cảm xúc rõ ràng hơn.
 - Giảm mơ hồ so với xét từ đơn: Ví dụ "thích" khi đi kèm "không" sẽ có nghĩa trái ngược hoàn toàn.
 - Tăng khả năng phân loại chính xác, đặc biệt trong các bài toán ngắn như bình luận YouTube.
- Nhược điểm:
- Tăng kích thước từ điển: Với N lớn, số lượng N-gram có thể tăng rất nhanh, dẫn đến vector biểu diễn thưa và nặng.
 - Dễ sinh nhiễu với N quá lớn: Các cụm từ hiếm có thể không có ý nghĩa hoặc gây rối cho mô hình.
 - Phụ thuộc vào cấu trúc câu: Nếu bình luận sai chính tả hoặc không rõ ràng, N-gram dễ bị nhiễu.

2.6. Giới thiệu về mô hình Hashing Vectorizer

2.6.1. Khái niệm

- Hashing Vectorizer là một phương pháp biểu diễn văn bản sử dụng hàm băm (hash function) để ánh xạ các từ thành các chỉ số trong một vector cố định kích thước. Khác với Count hay TF-IDF – vốn cần lưu từ điển (vocabulary) – phương pháp này không cần xây dựng từ điển thủ công, từ đó tiết kiệm bộ nhớ và tăng tốc độ xử lý.
- Trong bài toán phân loại cảm xúc bình luận YouTube, Hashing Vectorizer giúp mô hình xử lý nhanh và hiệu quả ngay cả khi dữ liệu có số lượng bình luận lớn và từ vựng phong phú, như các video có nhiều tương tác.

2.6.2. Nguyên lý hoạt động

- Với mỗi từ, áp dụng một hàm băm để xác định chỉ số trong vector.
 - Sau đó, cập nhật giá trị tại chỉ số đó bằng tần suất xuất hiện (giống Count) hoặc giá trị TF-IDF (tùy cấu hình).

- Vì nhiều từ có thể bị ánh xạ vào cùng một chỉ số (gọi là collision), Hashing Vectorizer không thể truy ngược lại từ gốc.
- Ví dụ:
 - Giả sử vector có 8 chiều.
 - Từ "tuyệt vời" được băm thành vị trí số 3 $\rightarrow$ vector[3] += 1
 - Từ "không thích" cũng băm thành số 3 $\rightarrow$ vector[3] += 1 (xảy ra collision)

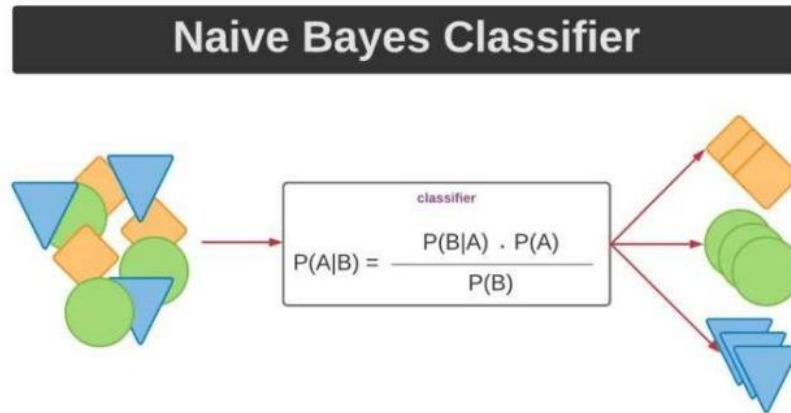
2.6.3. Ưu và nhược điểm:

- Ưu điểm:
 - Không cần từ điển: Giúp giảm đáng kể bộ nhớ và tiết kiệm thời gian tiền xử lý.
 - Xử lý nhanh với dữ liệu lớn: Phù hợp với các tập bình luận lớn như trên YouTube.
 - Ánh xạ vào vector có kích thước cố định: Dễ dàng tích hợp vào các mô hình học máy.
- Nhược điểm:
 - Không thể truy ngược từ: Vì sử dụng hàm băm nên không biết từ nào gây ra giá trị tại chỉ số đó.
 - Dễ xảy ra xung đột (collision): Nhiều từ có thể bị băm vào cùng một vị trí, gây nhiễu dữ liệu.
 - Không hỗ trợ phân tích sâu: Không cung cấp insight rõ ràng như TF-IDF về tầm quan trọng từ vựng.

2.7. Áp dụng các phương pháp học máy truyền thống trên biểu diễn

BoW

2.7.1. Thuật toán Naive Bayes (Naive Bayesian Classification)



Bảng 2. 4 Mô hình phân lớp NB

- Naive Bayes là một phương pháp phân loại dựa trên thống kê, vận dụng Định lý Bayes với giả định đơn giản rằng các đặc trưng (features) của dữ liệu là độc lập với nhau. Phương pháp này xuất phát từ ý tưởng rằng các văn bản thuộc các nhóm (lớp) khác nhau thường có những đặc trưng riêng biệt, ví dụ như tần suất xuất hiện của các từ, cụm từ, số lượng từ đặc trưng, hoặc các ký hiệu đặc biệt.
- Trong bài toán phân loại văn bản, các văn bản (document) được biểu diễn dưới dạng vector các đặc trưng, chẳng hạn như số lượng từ (Bag of Words) hoặc tần suất xuất hiện của từ (TF-IDF). Mô hình Naive Bayes sẽ học từ tập dữ liệu huấn luyện để phân loại các văn bản mới vào các lớp phù hợp (ví dụ: phân loại văn bản theo chủ đề, theo cảm xúc tích cực/tiêu cực, phân loại tin tức...).
- Định lý Bayes
 - Thuật toán Naive Bayes dựa trên định lý Bayes để tính toán xác suất của một lớp (C) dựa trên một văn bản (D) cho trước:

$$P(C|D) = \frac{P(D|C) \times P(C)}{P(D)}$$

Trong đó:

- $P(C|D)$: Xác suất văn bản D thuộc về lớp C .
- $P(D|C)$: Xác suất xuất hiện văn bản D khi biết rằng nó thuộc lớp C .
- $P(C)$: Xác suất tiên nghiệm (prior probability) của lớp C .
- $P(D)$: Xác suất xuất hiện văn bản D (có thể coi là hằng số khi so sánh giữa các lớp).

- Mô hình và cách huấn luyện
 - Trong phân loại văn bản, Multinomial Naive Bayes là biến thể phổ biến nhất, đặc biệt phù hợp với các đặc trưng là số đếm hoặc tần suất từ. Mô hình được huấn luyện trên tập dữ liệu đã được xử lý và gán nhãn sẵn (supervised learning), với mỗi văn bản có nhãn lớp cụ thể.
 - Mô hình sẽ học cách tính toán:
 - Xác suất từ xuất hiện trong mỗi lớp $P(\text{từ}|C)$.
 - Xác suất tiên nghiệm của mỗi lớp $P(C)$.
 → Sau khi được huấn luyện, mô hình có thể dự đoán lớp cho các văn bản mới bằng cách chọn lớp có xác suất cao nhất theo công thức trên.
- Ví dụ minh họa:
 - Giả sử cần phân loại cảm xúc của bình luận YouTube thành 3 nhóm: "Tích cực", "Tiêu cực" và "Trung lập".
 - Một bình luận như sau:
 - "Video này rất hay, nội dung hấp dẫn và ý nghĩa!"

Bước 1: Biểu diễn bình luận dưới dạng BoW

- Từ điển (Vocabulary): ["hay", "hấp dẫn", "ý nghĩa", "tệ", "chán", "bình thường"]

- Biểu diễn BoW của bình luận: [1, 1, 1, 0, 0, 0]

Bước 2: Xác suất xuất hiện của các từ trong từng lớp cảm xúc ($P(\text{Từ}|\text{C})$).

Giả sử từ "hay", "hấp dẫn", "ý nghĩa" thường xuất hiện nhiều trong các bình luận thuộc lớp "Tích cực".

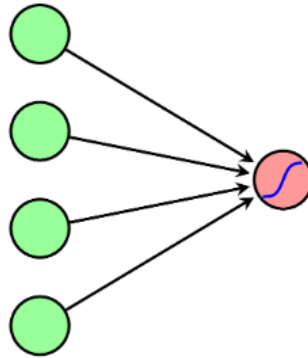
Bước 3: Tính xác suất của bình luận thuộc từng lớp

- $P(\text{Tích cực} | \text{bình luận}) = P(\text{hay}|\text{Tích cực}) \times P(\text{hấp dẫn}|\text{Tích cực}) \times P(\text{ý nghĩa}|\text{Tích cực}) \times P(\text{Tích cực})$
- $P(\text{Tiêu cực} | \text{bình luận}) = P(\text{hay}|\text{Tiêu cực}) \times P(\text{hấp dẫn}|\text{Tiêu cực}) \times P(\text{ý nghĩa}|\text{Tiêu cực}) \times P(\text{Tiêu cực})$
- $P(\text{Trung lập} | \text{bình luận}) = P(\text{hay}|\text{Trung lập}) \times P(\text{hấp dẫn}|\text{Trung lập}) \times P(\text{ý nghĩa}|\text{Trung lập}) \times P(\text{Trung lập})$

Bước 4: Phân loại: Do xác suất cao nhất thuộc về lớp "Tích cực", bình luận này sẽ được phân loại là Tích cực.

- Ưu điểm của Naive Bayes trong phân loại văn bản:
 - Hiệu quả và đơn giản, dễ triển khai.
 - Tốc độ nhanh, phù hợp với các tập dữ liệu lớn.
 - Có khả năng mở rộng, dễ dàng cập nhật mô hình khi có dữ liệu mới.
 - Hiệu quả cao trong các bài toán phân loại văn bản, nhất là khi các đặc trưng tách biệt rõ ràng giữa các lớp.
- Nhược điểm:
 - Phụ thuộc nhiều vào giả định độc lập giữa các từ, điều này có thể không hoàn toàn đúng với ngôn ngữ tự nhiên.
 - Kết quả bị ảnh hưởng nếu tập dữ liệu huấn luyện không đầy đủ hoặc có nhiễu.
 - Khó khăn khi phải xử lý các văn bản có chứa các từ hoặc dạng từ mới chưa từng xuất hiện trong dữ liệu huấn luyện (Out-of-vocabulary problem).

2.7.2. Thuật toán Logistic Regression trong phân loại văn bản



Logistic Regression

Bảng 2. 5 Mô hình hồi quy Logistic

- Logistic Regression (Hồi quy Logistic) là một kỹ thuật học máy phổ biến và hiệu quả, thường được sử dụng trong các bài toán phân loại, đặc biệt là phân loại văn bản. Dù có tên gọi là "hồi quy", mô hình này thực chất thực hiện nhiệm vụ phân loại tuyến tính, chủ yếu được áp dụng cho các bài toán phân loại hai lớp (binary classification) và cũng có thể mở rộng để xử lý các bài toán phân loại nhiều lớp (multi-class classification).
- Nguyên lý hoạt động
 - Logistic Regression sử dụng hàm sigmoid để mô hình hóa xác suất của một văn bản thuộc về một lớp cụ thể. Hàm sigmoid có dạng:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Trong đó $z = w^T x + b$, với:

- x : vector đặc trưng biểu diễn văn bản (thường là BoW, TF-IDF...).
- w : vector trọng số học được qua quá trình huấn luyện.
- b : hệ số điều chỉnh (bias).

- Cách huấn luyện và dự đoán

- Quá trình huấn luyện Logistic Regression tìm bộ trọng số w và b tối ưu bằng cách tối thiểu hóa hàm mất mát (loss function), thường sử dụng hàm mất mát log-loss (cross-entropy): Mô hình sẽ học cách tính toán:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Trong đó:

- N : số lượng mẫu huấn luyện.
- y_i : nhãn thực tế của văn bản thứ i .
- $\hat{y}_i$: xác suất dự đoán.

- Ví dụ minh họa

- Giả sử ta có một bình luận từ YouTube:
 - "Video này rất hay, nội dung cuốn hút!"

Bước 1: Biểu diễn văn bản

- Từ điển quan trọng: $["hay", "nội dung", "cuốn hút"]$
- Biến văn bản thành vector đặc trưng bằng BoW hoặc TF-IDF.

Bước 2: Áp dụng Logistic Regression: Mô hình tính tổng trọng số của các từ và đưa vào hàm sigmoid để dự đoán xác suất:

- $P(\text{Tích cực} \mid \text{bình luận}) = 0.88$
- $P(\text{Trung lập} \mid \text{bình luận}) = 0.1$
- $P(\text{Tiêu cực} \mid \text{bình luận}) = 0.02$

→ Kết quả: Bình luận này được phân loại là Tích cực do xác suất cao nhất.

- Ưu điểm của Logistic Regression trong phân loại văn bản:

- Đơn giản, dễ hiểu và dễ triển khai.

- Hiệu quả cao với các bài toán phân loại tuyến tính.
 - Ổn định khi xử lý dữ liệu lớn và nhiễu, nhất là khi các đặc trưng đã được chuẩn hóa.
 - Không yêu cầu giả định độc lập giữa các đặc trưng, khác với Naive Bayes.
- Nhược điểm:
- Khó mở rộng cho các quan hệ phi tuyến tính trừ khi kết hợp với các kỹ thuật biến đổi đặc trưng.
 - Hiệu quả kém nếu dữ liệu không thể phân tách tuyến tính.
 - Cần xử lý tốt các vấn đề như đa cộng tuyến (collinearity) giữa các đặc trưng.

2.7.3. Thuật toán *Decision Tree* trong phân loại văn bản

Decision and Classification Trees...



Bảng 2. 6 Mô hình phân lớp DT

- Decision Tree (Cây quyết định) là một thuật toán phân loại dễ hiểu và rất hiệu quả, hoạt động dựa trên nguyên lý chia tách dữ liệu theo các đặc trưng để xây dựng một cây gồm các nút quyết định và nhánh phân chia. Trong bài toán phân loại văn bản, cây quyết định sử dụng các đặc trưng như từ khóa, tần suất từ, hoặc các chỉ số TF-IDF để chia dữ liệu thành các nhóm văn bản cùng nhãn.
- Nguyên lý hoạt động:
 - Cây quyết định được hình thành thông qua việc chọn lựa các đặc trưng phân chia dữ liệu sao cho tối ưu nhất, dựa trên các tiêu chí như Entropy,

Information Gain, hoặc Gini Index. Quá trình xây dựng cây diễn ra theo các bước sau:

- Chọn đặc trưng phân chia: Dựa vào tiêu chí đánh giá (ví dụ: chọn đặc trưng có Information Gain cao nhất).
 - Tạo nút quyết định: Sau khi chọn được đặc trưng, tạo ra một nút trong cây phân chia dữ liệu.
 - Phân chia dữ liệu: Dữ liệu được phân chia theo các giá trị của đặc trưng, và quá trình tiếp tục với các tập con.
 - Điều kiện dừng: Quá trình phân chia dừng lại khi các nút chứa dữ liệu có nhãn đồng nhất, hoặc khi đạt được các điều kiện dừng như độ sâu cây tối đa hay số lượng mẫu tối thiểu.
- Công thức tiêu biểu - Information Gain:
- Quá trình huấn luyện Logistic Regression tìm bộ trọng số w và b tối ưu bằng cách tối thiểu hóa hàm mất mát (loss function), thường sử dụng hàm mất mát log-loss (cross-entropy): Mô hình sẽ học cách tính toán:

$$IG(D, A) = Entropy(D) - \sum_{v \in Values(A)} \frac{|D_v|}{|D|} \times Entropy(D_v)$$

Trong đó:

- D : tập dữ liệu hiện tại.
- A : đặc trưng được chọn.
- D_v : tập con của D với giá trị đặc trưng $A = v$.

- Ví dụ minh họa:
- Xét một bình luận:
"Âm thanh tệ quá, không nghe rõ gì cả!"

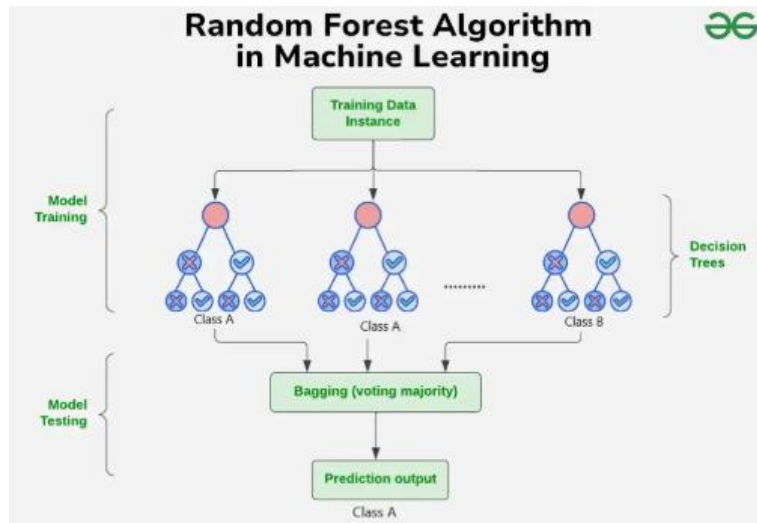
Bước 1: Xây dựng cây quyết định: Thuật toán chọn đặc trưng để phân tách bình luận, dựa vào các từ khóa quan trọng:

- Bình luận có chứa từ "tệ", "không" hoặc "nghe rõ"? → **Có** → Phân loại **Tiêu cực**
- Nếu không, bình luận có chứa từ "tuyệt vời", "hay" hoặc "xuất sắc"? → **Có** → Phân loại **Tích cực**
- Nếu không, gán nhãn **Trung lập**.

Bước 2: Phân loại bình luận trên: Bình luận chứa từ "tệ" và "không nghe rõ" → Phân loại là **Tiêu cực**.

- Ưu điểm:
 - Dễ hiểu, trực quan, dễ giải thích cho con người.
 - Không yêu cầu giả định về mối quan hệ giữa các đặc trưng (phi tuyến tính).
 - Xử lý tốt dữ liệu không tuyến tính và dễ mở rộng cho phân loại đa lớp.
 - Có thể kết hợp với kỹ thuật ensemble (rừng ngẫu nhiên, boosting) để nâng cao hiệu quả.
- Nhược điểm:
 - Dễ bị overfitting nếu không kiểm soát độ sâu của cây.
 - Nhạy cảm với dữ liệu nhiễu, các đặc trưng không quan trọng có thể làm cây phân tách sai.
 - Khó khăn khi xử lý dữ liệu có nhiều đặc trưng hiếm gặp hoặc dữ liệu không cân bằng.
 - Cần xử lý tốt các vấn đề như đa cộng tuyến (collinearity) giữa các đặc trưng.

2.7.4. Thuật toán Random Forest



Bảng 2. 7 Thuật toán Random Forest

- Định nghĩa:
 - Random Forest là một thuật toán học máy mạnh mẽ được sử dụng cho cả phân loại và hồi quy. Nó là sự kết hợp của nhiều cây quyết định (Decision Trees) để tạo ra một mô hình dự đoán chính xác hơn. Điểm đặc biệt của Random Forest là việc sử dụng nhiều cây quyết định trong quá trình huấn luyện, từ đó đưa ra kết quả dự đoán dựa trên các dự đoán của từng cây. Kết quả cuối cùng của mô hình được tính bằng cách lấy trung bình (đối với hồi quy) hoặc bỏ phiếu đa số (đối với phân loại).
- Ý nghĩa và ứng dụng:
 - Random Forest có nhiều ưu điểm, như khả năng xử lý tốt cả dữ liệu phân loại và dữ liệu số, độ chính xác cao, giảm thiểu hiện tượng overfitting và có thể xử lý dữ liệu bị thiếu. Nó được ứng dụng rộng rãi trong nhiều lĩnh vực như y tế (dự đoán bệnh), tài chính (phát hiện gian lận), marketing (dự đoán hành vi khách hàng), và các hệ thống máy tự động.
- Nguyên lý hoạt động

- Quá trình xây dựng mô hình Random Forest bao gồm hai bước quan trọng: Bootstrap Sampling và Feature Randomness.
 - Bootstrap Sampling: Mỗi cây quyết định trong rừng được xây dựng từ một tập con của dữ liệu gốc, được chọn ngẫu nhiên với phép thay thế. Điều này giúp tăng sự đa dạng giữa các cây.
 - Feature Randomness: Tại mỗi nút phân chia của cây, chỉ một số thuộc tính ngẫu nhiên được chọn để quyết định cách chia tách dữ liệu. Điều này giúp giảm hiện tượng overfitting và giảm tác động của các đặc trưng không quan trọng.
- Sau khi huấn luyện, kết quả dự đoán được tính dựa trên tổng hợp các cây thành viên:
 - Bỏ phiếu đa số (Majority Voting): Trong bài toán phân loại, kết quả cuối cùng sẽ là lớp có nhiều phiếu nhất từ các cây.
 - Trung bình giá trị dự đoán (Averaging): Trong bài toán hồi quy, kết quả cuối cùng sẽ là trung bình giá trị dự đoán của tất cả các cây trong rừng.
- Công thức tiêu biểu
 - Công thức phân loại: Mô tả cách Random Forest tổng hợp kết quả từ nhiều cây quyết định để đưa ra dự đoán cuối cùng thông qua phương pháp bỏ phiếu đa số.
 - Công thức hồi quy: Trong trường hợp hồi quy, công thức tính trung bình giá trị dự đoán từ tất cả các cây, giúp làm cho mô hình trở nên ổn định hơn và giảm thiểu tác động của nhiễu.

- Bài toán phân loại:

$$\hat{y} = \arg \max_c \sum_{t=1}^T 1(h_t(x) = c)$$

Trong đó, $h_t(x)$ là dự đoán của cây thứ t và $1(.)$ là hàm chỉ thị.

- Bài toán hồi quy:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T h_t(x)$$

- MSE: Được sử dụng để đo lường mức độ sai lệch giữa giá trị thực tế và giá trị dự đoán của mô hình.
- Gini Impurity và Entropy: Hai chỉ số này giúp xác định mức độ hỗn loạn trong dữ liệu tại mỗi nút của cây quyết định, ảnh hưởng trực tiếp đến cách cây được xây dựng.

- MSE (Mean Squared Error) cho hồi quy:

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- Gini Impurity hoặc Entropy cho phân loại:

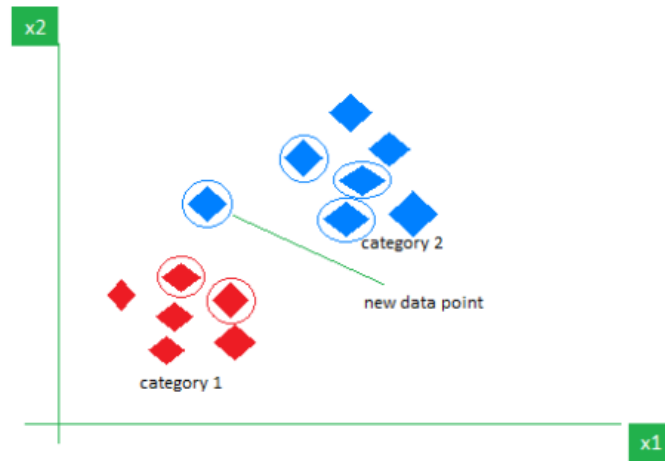
$$Gini = 1 - \sum_{i=1}^C p_i^2$$

$$Entropy = - \sum_{i=1}^C p_i \log p_i$$

- Cơ chế tối ưu: Random Forest áp dụng một số kỹ thuật tối ưu để cải thiện hiệu suất mô hình:
 - Giới hạn độ sâu của cây: Việc này giúp giảm hiện tượng overfitting bằng cách kiểm soát độ phức tạp của mỗi cây quyết định.

- Chọn số lượng cây hợp lý: Mô hình trở nên ổn định hơn khi số lượng cây tăng, tuy nhiên điều này cũng đồng nghĩa với việc tốn nhiều tài nguyên tính toán.
 - Điều chỉnh số lượng đặc trưng được chọn: Việc này giúp giảm sự tương quan giữa các cây và cải thiện hiệu quả hoạt động của mô hình tổng thể.
 - Xử lý dữ liệu thiếu: Random Forest có khả năng tự động điền giá trị thiếu bằng cách dự đoán từ các cây khác.
- Ưu và nhược điểm
- Ưu điểm:
 - Giảm hiện tượng overfitting so với khi chỉ sử dụng một cây quyết định đơn lẻ.
 - Hiệu suất cao và có thể áp dụng với nhiều loại dữ liệu khác nhau.
 - Xử lý tốt các dữ liệu có nhiễu nhờ việc tổng hợp kết quả từ nhiều cây quyết định.
 - Không yêu cầu chuẩn hóa dữ liệu đầu vào, có thể làm việc với cả dữ liệu phân loại và dữ liệu liên tục.
 - Hỗ trợ việc xử lý dữ liệu thiếu mà không cần phải loại bỏ các mẫu dữ liệu.
 - Nhược điểm:
 - Tốc độ huấn luyện chậm hơn so với một cây quyết định vì cần tạo nhiều cây.
 - Tốn tài nguyên tính toán khi số lượng cây lớn.
 - Khó diễn giải hơn so với một cây quyết định đơn lẻ.

2.7.5. Thuật toán KNN (K-Nearest Neighbors)



Bảng 2. 8 KNN (K-Nearest Neighbors)

- Định nghĩa:
 - KNN (K-Nearest Neighbors) là một thuật toán học máy có giám sát, chủ yếu được áp dụng trong các bài toán phân loại và hồi quy. Thuật toán này thuộc vào nhóm **lazy learning** (học thụ động), có nghĩa là nó không học từ dữ liệu huấn luyện mà chỉ thực hiện tính toán khi cần dự đoán cho một điểm dữ liệu mới. Khi muốn phân loại một điểm dữ liệu mới, KNN tìm K điểm gần nhất trong tập huấn luyện và xác định nhãn của điểm mới dựa vào số lượng nhãn xuất hiện nhiều nhất trong K điểm lân cận.
- Ý nghĩa và ứng dụng:
 - Nhận diện chữ viết tay: Sử dụng trong các bài toán nhận diện chữ viết tay như phân loại các chữ số trong hệ thống nhận diện chữ viết tay (ví dụ, MNIST dataset).
 - Dự đoán bệnh: Dự đoán các bệnh lý từ dữ liệu y tế như trong phân tích kết quả xét nghiệm máu, các chỉ số sức khỏe,...
 - Phát hiện gian lận: Phát hiện gian lận trong giao dịch thẻ tín dụng bằng cách phân tích các giao dịch và nhận diện những hành vi bất thường.

- Hệ thống gợi ý sản phẩm: Được sử dụng trong các hệ thống gợi ý (recommender system), ví dụ như gợi ý sản phẩm hoặc bài viết cho người dùng trên nền tảng thương mại điện tử hay mạng xã hội.
- Nguyên lý hoạt động:
 - Chọn tham số K (số lượng hàng xóm gần nhất).
 - Tính khoảng cách giữa điểm cần phân loại và tất cả các điểm trong tập huấn luyện. Các phương pháp tính khoảng cách phổ biến bao gồm:
 - Khoảng cách Euclidean: Là phương pháp phổ biến nhất để tính khoảng cách giữa các điểm trong không gian n chiều.
 - Khoảng cách Manhattan: Tính tổng của các hiệu giữa các tọa độ tương ứng.
 - Khoảng cách Minkowski: Một tổng quát của khoảng cách Euclidean và Manhattan, có thể điều chỉnh bằng một tham số p.
 - Chọn K điểm gần nhất.
 - Xác định nhãn dự đoán:
 - Phân loại: Nhãn của điểm dữ liệu được xác định là nhãn xuất hiện nhiều nhất trong K điểm gần nhất.
 - Hồi quy: Giá trị của điểm dự đoán là trung bình giá trị của K điểm gần nhất.
- Công thức tiêu biểu:
 - Khoảng cách Euclidean là công thức phổ biến nhất được sử dụng trong thuật toán KNN để đo độ tương đồng giữa hai điểm trong không gian dữ liệu. Công thức tính khoảng cách Euclidean giữa hai điểm (x_1, y_1) và (x_2, y_2) trong không gian 2 chiều được tính như sau:

Khoảng cách Euclidean (thường dùng nhất)

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Với:

- $A(x_1, y_1)$ và $B(x_2, y_2)$ là hai điểm trong không gian dữ liệu.
- Công thức này dễ hiểu và phản ánh chính xác độ tương đồng giữa hai điểm.

- Công thức này tính toán độ dài đường thẳng ngắn nhất nối hai điểm trong không gian.
- Nó giúp xác định mức độ tương đồng giữa hai đối tượng dựa trên tọa độ của chúng.
- Trong KNN, khoảng cách này được dùng để tìm k điểm gần nhất với điểm cần phân loại.
- Công thức dự đoán giá trị trung bình trong hồi quy thể hiện cách dự đoán giá trị trung bình có trọng số trong bài toán hồi quy sử dụng thuật toán KNN.

Dự đoán giá trị trung bình (trong hồi quy)

$$\hat{y} = \frac{\sum_{i=1}^K w_i y_i}{\sum_{i=1}^K w_i}$$

- Trọng số càng lớn, giá trị càng có ảnh hưởng đến kết quả dự đoán.

- Khi áp dụng KNN vào bài toán hồi quy, thay vì dự đoán nhãn của điểm dữ liệu mới dựa trên số đông như trong phân loại, ta dự đoán giá trị đầu ra bằng cách lấy trung bình có trọng số của các giá trị $y_{i1}, y_{i2}, \dots, y_{ik}$ của k điểm lân cận gần nhất.
- Trọng số w_i xác định mức độ ảnh hưởng của từng điểm lân cận. Điểm nào gần hơn sẽ có trọng số lớn hơn, ảnh hưởng nhiều hơn đến kết quả dự đoán.

- Cơ chế tối ưu
 - Chọn K hợp lý:
 - K quá nhỏ → dễ bị nhiễu, overfitting.
 - K quá lớn → có thể mất chi tiết, underfitting.
 - Chuẩn hóa dữ liệu: Tránh tình trạng các thuộc tính có đơn vị đo khác nhau ảnh hưởng đến kết quả.
 - Dùng cấu trúc dữ liệu tối ưu (KD-tree, Ball-tree) để tăng tốc độ tìm kiếm hàng xóm.
- Ưu và nhược điểm
 - Ưu điểm:
 - Đơn giản, dễ hiểu, dễ triển khai: KNN là một thuật toán trực quan và dễ triển khai mà không yêu cầu quá nhiều kiến thức về lý thuyết học máy.
 - Không cần giai đoạn huấn luyện phức tạp: Vì là thuật toán "lazy learning", KNN không cần một quá trình huấn luyện phức tạp như các thuật toán học máy khác. Việc học chỉ diễn ra khi cần phân loại một điểm dữ liệu mới.
 - Xử lý tốt với dữ liệu nhiễu: KNN có khả năng xử lý dữ liệu nhiễu bằng cách dựa vào sự đồng nhất của các điểm gần nhất. Dữ liệu nhiễu sẽ ít ảnh hưởng vì quyết định cuối cùng được đưa ra từ nhóm điểm gần nhất.
 - Nhược điểm:
 - Chậm với tập dữ liệu lớn: KNN yêu cầu tính toán khoảng cách giữa điểm dữ liệu cần phân loại và tất cả các điểm trong tập huấn luyện. Điều này dẫn đến độ phức tạp tính toán cao và chậm khi làm việc với tập dữ liệu lớn.
 - Nhạy cảm với dữ liệu không cân bằng: Nếu các lớp trong dữ liệu không được phân bố đều (dữ liệu không cân bằng), KNN có thể bị lệch về phía lớp có nhiều điểm dữ liệu hơn.

- Cần chuyển đổi dữ liệu số sang dạng phù hợp nếu làm việc với dữ liệu định tính: KNN yêu cầu dữ liệu đầu vào phải có thể tính được khoảng cách, điều này có thể gặp khó khăn nếu làm việc với dữ liệu định tính hoặc dữ liệu không phải dạng số (ví dụ: chuỗi ký tự, danh mục).

2.8. Áp dụng các phương pháp học máy truyền thống bài toán phân loại cảm xúc bình luận trên YouTube

2.8.1. Mục tiêu của bài toán:

- Xây dựng mô hình phân loại cảm xúc bình luận trên YouTube thành ba nhóm: Tích cực (Positive), Tiêu cực (Negative) và Trung lập (Neutral).
- Phương pháp thực hiện:
 - Thu thập dữ liệu bình luận từ YouTube bằng Selenium, lưu vào file comments.csv.
 - Gán nhãn cảm xúc cho từng bình luận bằng API Gemini (Google Generative AI).
 - Sử dụng phương pháp biểu diễn văn bản: Bag of Words (CountVectorizer), TF-IDF (TfidfVectorizer).
 - Huấn luyện mô hình phân loại cảm xúc bằng các thuật toán học máy truyền thống: Logistic Regression, Naive Bayes...
 - Đánh giá hiệu suất mô hình dựa trên các chỉ số: Accuracy, Precision, Recall, F1-score.
- Dữ liệu đầu vào: File comments.csv gồm nội dung bình luận thô.
- Dữ liệu đầu ra: File comments_labeled.csv gồm bình luận và nhãn cảm xúc tương ứng.

2.8.2. Nội dung các bước thực hiện chi tiết

- Trước khi trích xuất đặc trưng, việc **làm sạch dữ liệu văn bản** là bước cần thiết nhằm giảm nhiễu và cải thiện chất lượng của các đặc trưng được trích xuất. **Các bước tiền xử lý bao gồm:**
 - Loại bỏ HTML và URL: Sử dụng các thư viện xử lý văn bản (ví dụ, BeautifulSoup) để loại bỏ các thẻ HTML và chuỗi URL không cần thiết.
 - Chuyển về chữ thường: Điều này giúp thống nhất cách biểu diễn từ, tránh tình trạng cùng một từ bị phân biệt do khác biệt chữ hoa và chữ thường.
 - Loại bỏ số và dấu câu: Các ký tự số và dấu câu thường không đóng góp nhiều thông tin ngữ nghĩa cho bài toán phân loại cảm xúc.
 - Xóa khoảng trắng thừa: Giúp chuẩn hóa dữ liệu văn bản sau các bước xử lý khác.
 - Tokenization và loại bỏ stopwords: Tách văn bản thành các đơn vị từ (tokens) và loại bỏ các từ dừng (stopwords) không có giá trị phân biệt.
- Những bước tiền xử lý này được thực hiện bằng cách sử dụng các thư viện Python như **nlTK**, **re** và **BeautifulSoup** với các hàm chuyên dụng. Các tham số mặc định của các hàm tiền xử lý thường dựa trên các tập từ dừng có sẵn và các biểu thức chính quy để xử lý số, ký tự đặc biệt.

```
import re
import nltk
from bs4 import BeautifulSoup
```

- Sau khi làm sạch dữ liệu, các phương pháp biểu diễn văn bản dưới dạng vector được áp dụng nhằm chuyển đổi thông tin từ dạng chuỗi sang dạng số để các thuật toán học máy có thể xử lý. Hai phương pháp chính được sử dụng là:

BoW và TF-IDF:

- **Bag-of-Words (BoW):** Nhóm sử dụng CountVectorizer để xây dựng ma trận đặc trưng từ dữ liệu văn bản:

```
from sklearn.feature_extraction.text import
CountVectorizer
```

- **TF-IDF:** áp dụng TfidfVectorizer với các tham số mặc định của scikit-learn, cho phép tự động chuẩn hóa và tính toán giá trị TF-IDF cho từng tài liệu.

```
from sklearn.feature_extraction.text import
TfidfVectorizer
```

- **Xây dựng và hiệu chỉnh các mô hình học máy:** Sau khi biểu diễn văn bản dưới dạng vector, tập dữ liệu được chia thành tập huấn luyện và tập kiểm tra thông qua hàm *train_test_split*. Các mô hình phân loại sau đây được áp dụng:

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

- **Naive Bayes:**

```
nb_model = MultinomialNB()
```

- Tham số: Sử dụng tham số mặc định.

- **Logistic Regression:**

```
lr_model = LogisticRegression(max_iter=1000)
```

- Tham số: Tham số *max_iter* được điều chỉnh (*max_iter*=1000)

- **k-Nearest Neighbors (k-NN):**

```
knn_model = KNeighborsClassifier(n_neighbors=5)
```

- Tham số: Số lượng hàng xóm k với giá trị k=5 được sử dụng.

- **Decision Tree:**

```
dt_model = DecisionTreeClassifier()
```

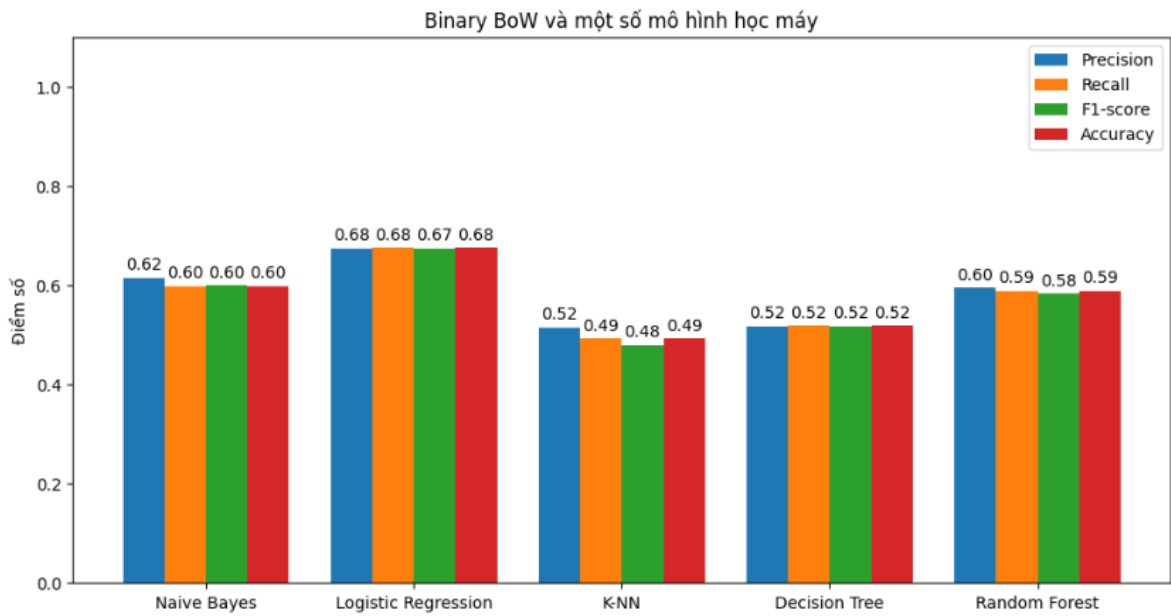
- Tham số: sử dụng tham số mặc định

- **Random Forest:**

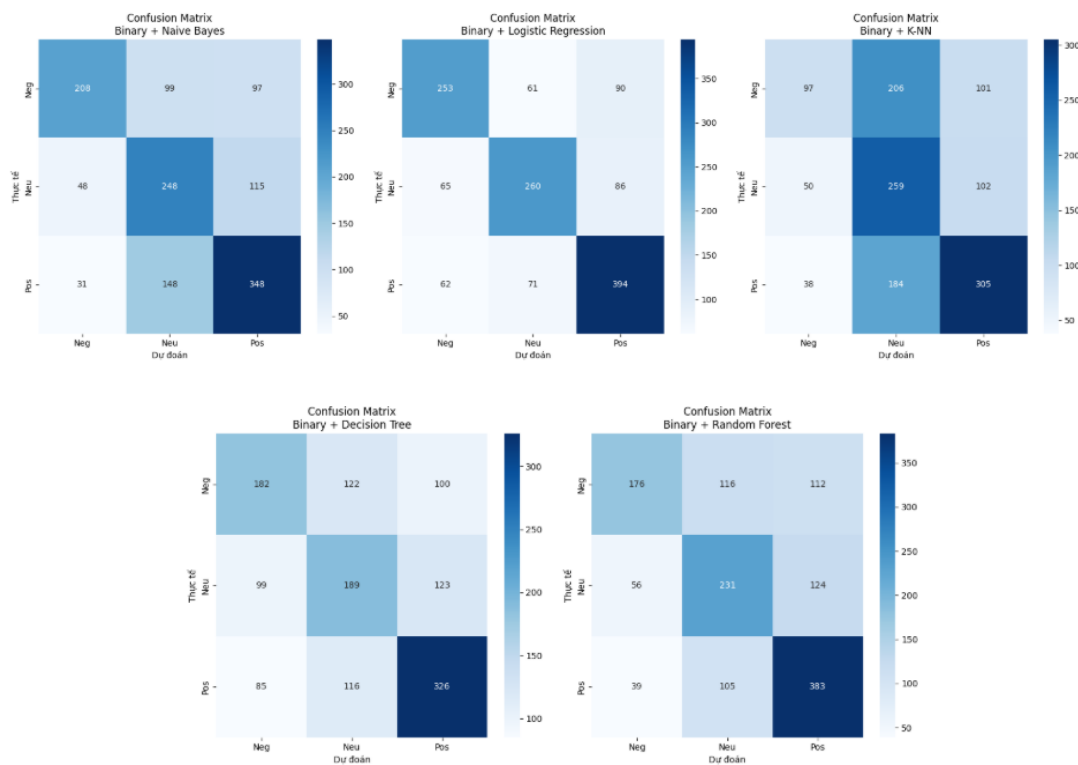
```
rf_model = RandomForestClassifier(random_state=42)
```

Tham số: điều chỉnh Số lượng cây trong rừng (`n_estimators=100`) và giá trị ngẫu nhiên (`random_state=42`).

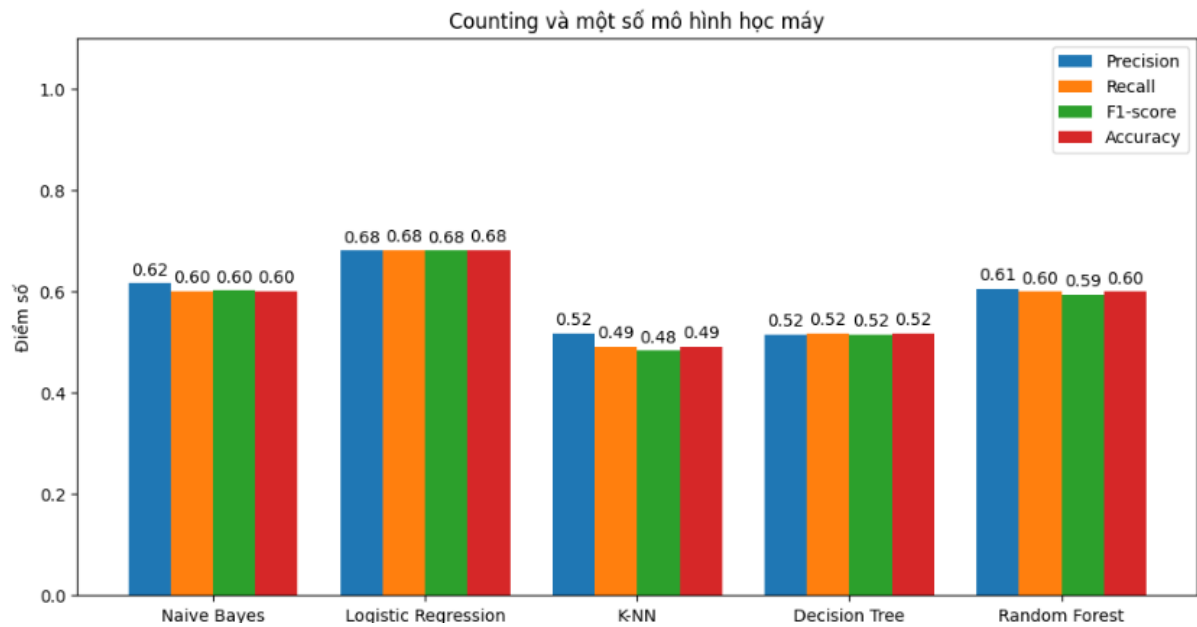
- **Đánh giá mô hình:** Để so sánh hiệu quả của các mô hình, các chỉ số đánh giá bao gồm độ chính xác (accuracy), precision, recall và F1-score một cách trực quan, nhóm sử dụng:
 - Biểu đồ cột: trực quan hóa các chỉ số đánh giá của từng mô hình theo nhóm (precision, recall, F1-score, accuracy).
 - Ma trận nhầm lẫn (Confusion Matrix): đánh giá chi tiết hiệu suất của các mô hình phân loại, cho biết số lượng dự đoán đúng và sai của mỗi lớp, từ đó giúp xác định các lỗi đặc trưng trong phân loại.
- **Kết quả và đánh giá chung mô hình:**



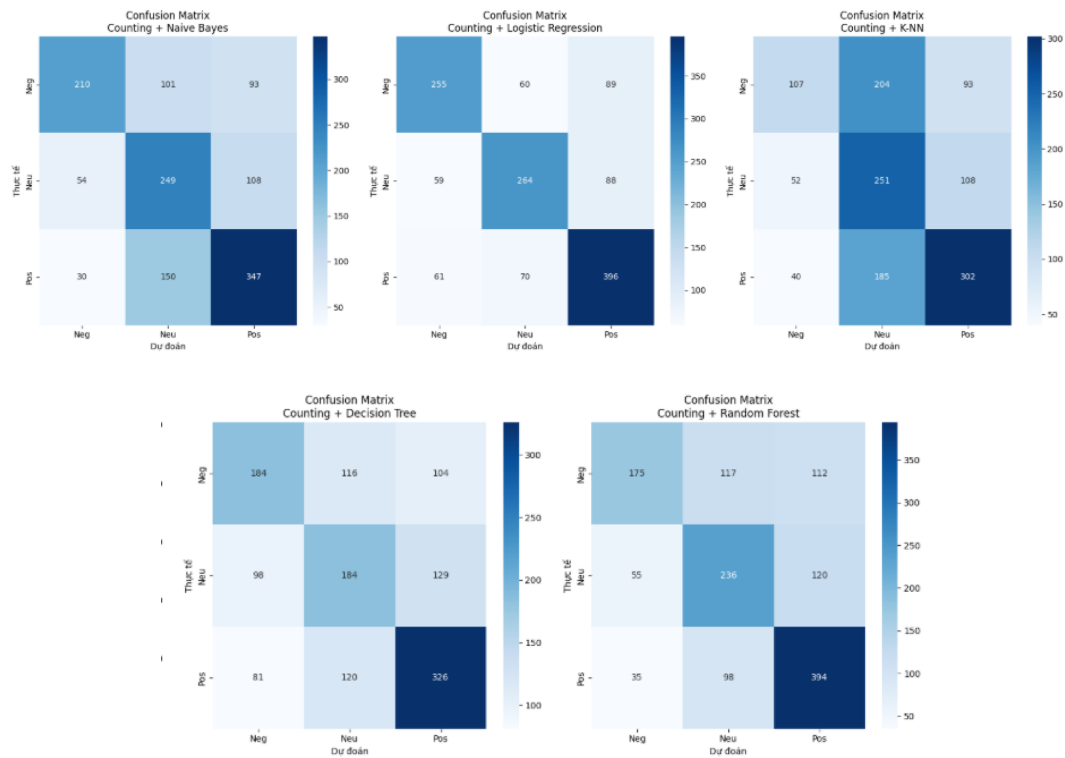
Hình 2. 1 Kết quả biểu đồ cột Binary



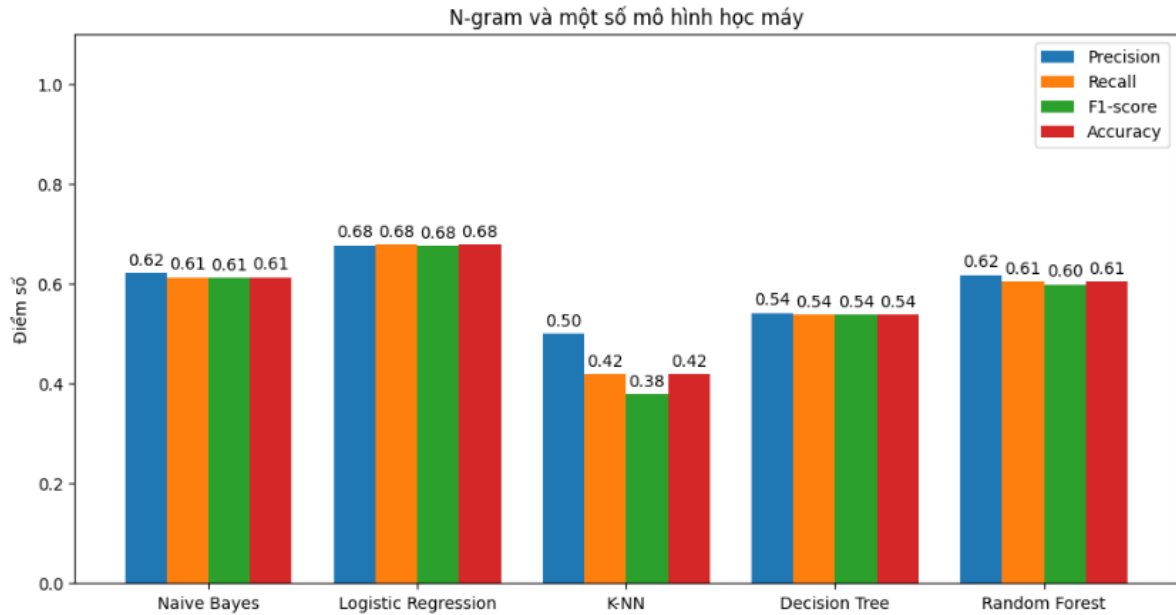
Hình 2. 2 Kết quả Confusion Matrix Binary



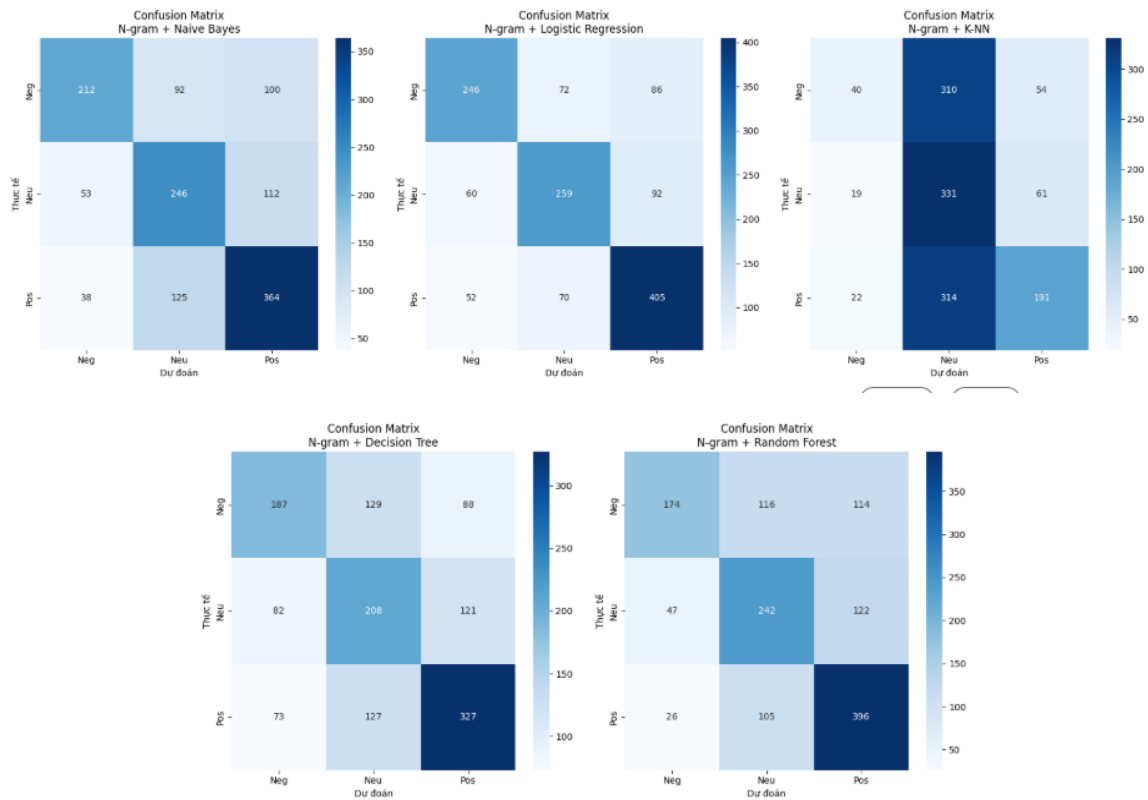
Hình 2. 3 Kết quả biểu đồ cột Counting



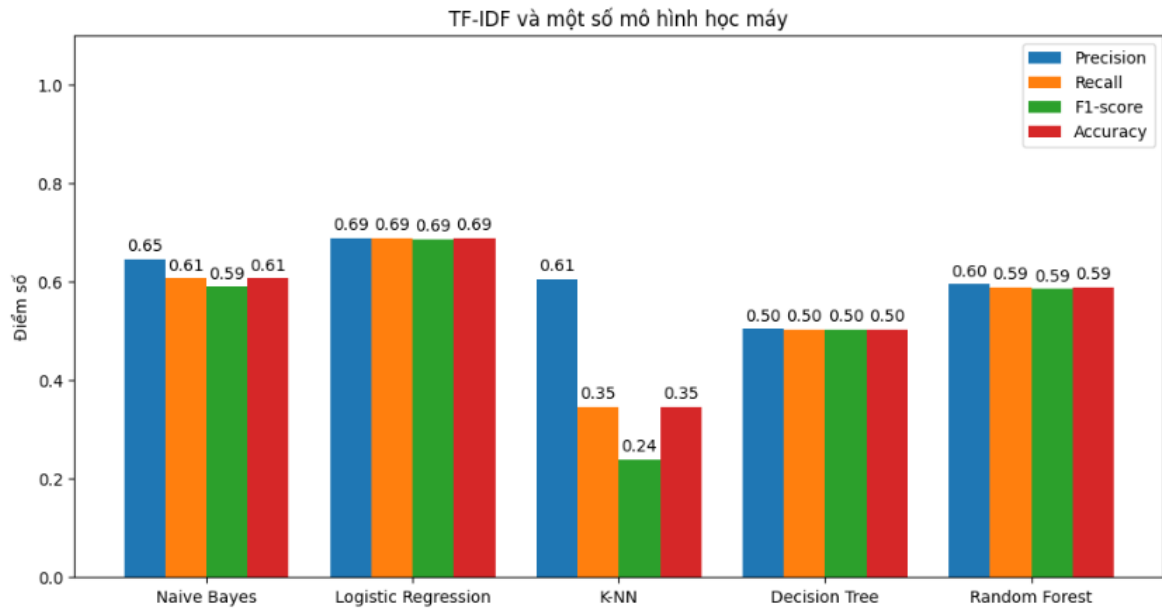
Hình 2. 4 Kết quả Confusion Matrix Counting



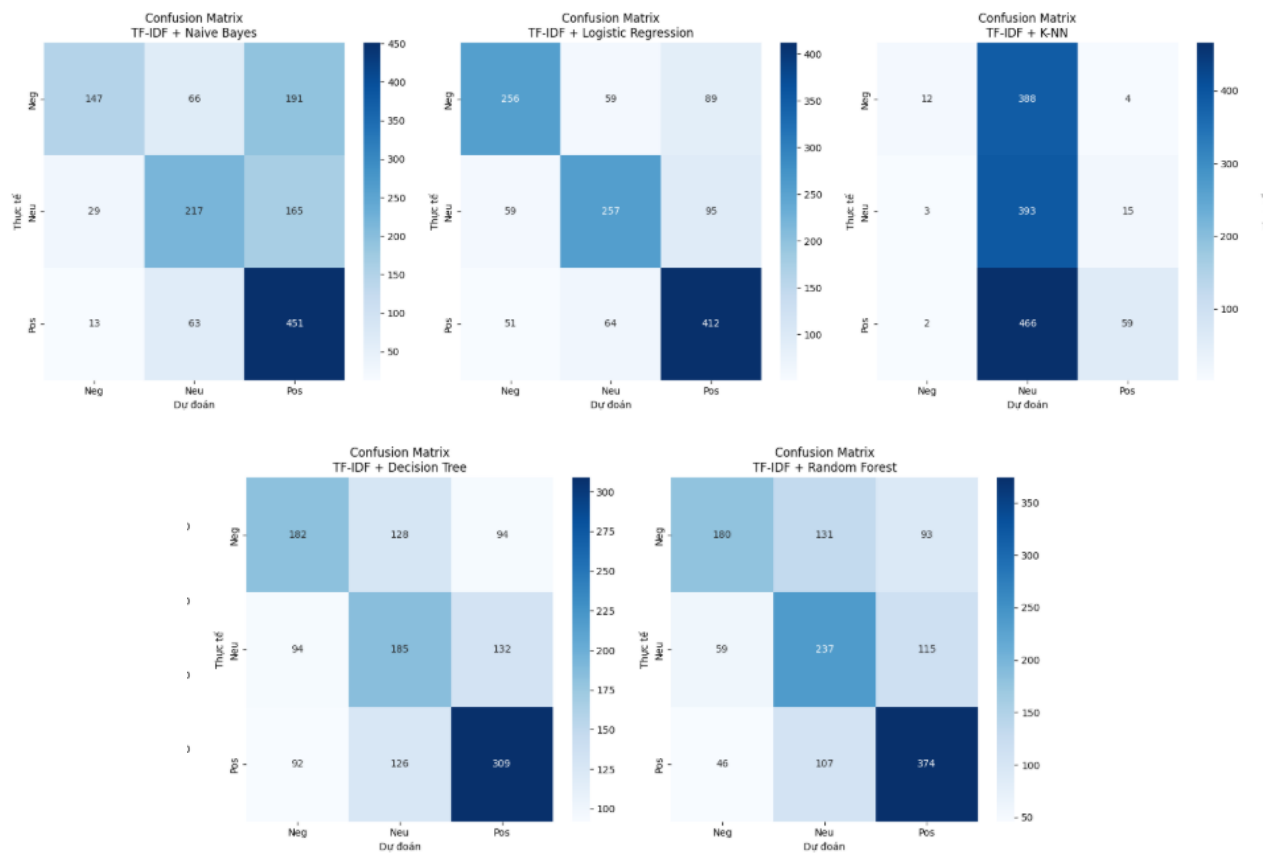
Hình 2. 5 Kết quả biểu đồ cột N-gram



Hình 2. 6 Kết quả Confusion Matrix N-gram



Hình 2. 7 Kết quả biểu đồ cột TF-IDF



Hình 2. 8 Kết quả Confusion Matrix TF-IDF

- Kết quả trực quan từ biểu đồ, nhóm thấy được mô hình hiện có độ chính xác tương đối ổn định (68% với mô hình học máy Logistic Regression).
- Thấp nhất với mô hình học máy K-NN.
- Nguyên nhân chủ yếu:
 - Từ chất lượng gắn nhãn: việc gắn nhãn tự động có thể đã làm một số bình luận bị sai lệch về nghĩa. Ví dụ:
 - “Ai còn nghe không” gắn nhãn “Pos”
 - Trong khi đó “Bài này ai còn xem không” lại được gắn nhãn là “Neu” khiến dữ liệu bị nhiễu gây ra sự sụt giảm độ chính xác của mô hình.
 - Dữ liệu bị trùng lặp và spam: Quá trình thu thập dữ liệu tự động trên nền tảng YouTube có thể thu được nhiều nội dung spam, trong đó có các bình luận lệch lạc hoặc trùng lặp. Dữ liệu nhiễu này ảnh hưởng tiêu cực đến quá trình huấn luyện mô hình, làm giảm khả năng tổng quát hóa và chất lượng dự đoán.

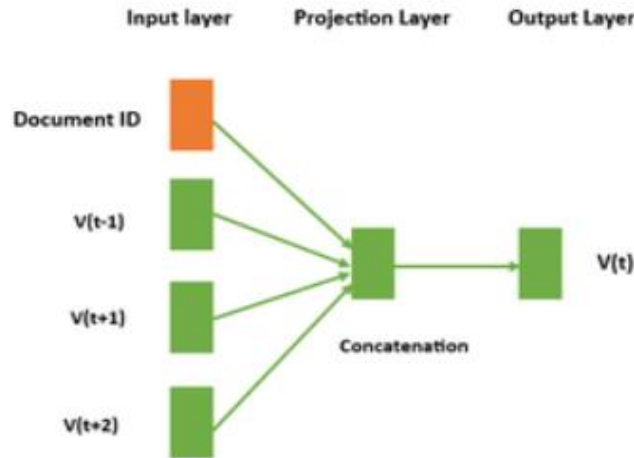
➤ **Đánh giá chung:**

- Phương pháp học máy Logistic Regression luôn giữ được độ chính xác tương đối và bền vững qua nhiều cách BoW khác nhau tận dụng khả năng biểu diễn trọng số của các từ quan trọng từ văn bản. Với các kỹ thuật regularization (L1, L2), Logistic Regression giúp giảm thiểu overfitting và cho kết quả ổn định qua nhiều cách biểu diễn (68~69%)
- Trong khi đó K-NN lại phụ thuộc vào “hàng xóm”, nhạy cảm với nhiễu, làm cho khoảng cách giữa các điểm trở nên đồng đều, từ đó làm giảm sức mạnh phân biệt của khoảng cách. Điều này dẫn đến việc K-NN khó xác định được hàng xóm thực sự có ý nghĩa, dẫn đến chất lượng dự đoán kém.
- Cải tiến:

- Làm sạch dữ liệu kĩ hơn trước khi huấn luyện mô hình, bao gồm: Xử lý lại quá trình gán nhãn không nhất quán để giảm sự không nhất quán, loại bỏ dữ liệu trùng lặp và các bình luận spam không liên quan để giảm nhiễu.
- Tối ưu hóa tham số cho Logistic Regression, điều chỉnh các hệ số regularization để duy trì hoặc cải thiện hiệu suất.
- Do đặc thù của dữ liệu thu thập tự động từ YouTube, nên nhóm cần xem lại quá trình giám sát chặt chẽ, cập nhật và kiểm tra định kỳ chất lượng của dữ liệu gán nhãn, thống nhất nhãn để đảm bảo mô hình luôn được học trên những dữ liệu sát với thực tế.

CHƯƠNG 3 – HUẤN LUYỆN MÔ HÌNH HỌC MÁY

3.1. Giới thiệu về Doc2Vec



Hình 3. 1 Mô hình Doc2Vec

Doc2Vec là một mô hình mở rộng từ Word2Vec, nhằm biểu diễn toàn bộ văn bản (document) dưới dạng vector liên tục (dense vector), thay vì chỉ biểu diễn từ đơn lẻ. Trong bài toán phân loại cảm xúc bình luận trên YouTube, Doc2Vec giúp mã hóa các bình luận thành vector số học, giữ lại được ngữ nghĩa tổng thể, bối cảnh và thông tin quan trọng của từng bình luận.

3.2. Các phiên bản mô hình Doc2Vec

- Doc2Vec có hai biến thể chính:
 - Distributed Memory version of Paragraph Vector (PV-DM):
 - Hoạt động giống như phương pháp Continuous Bag of Words (CBOW) trong Word2Vec.
 - Dùng thông tin từ các từ xung quanh (bối cảnh) và từ vector của văn bản để dự đoán từ tiếp theo.
 - Ưu điểm: Hiểu được ngữ cảnh và trật tự từ, biểu diễn tốt hơn về mặt ngữ nghĩa.

- Distributed Bag of Words version of Paragraph Vector (PV-DBOW):
 - Gần giống với Skip-gram trong Word2Vec.
 - Chỉ sử dụng vector của văn bản để dự đoán các từ trong văn bản, không cần nhớ toàn bộ từ vựng.
 - Ưu điểm: Nhanh hơn, tốn ít bộ nhớ hơn.
- Kết hợp cả hai phương pháp PV-DM và PV-DBOW để đạt hiệu quả cao.

3.3. Nguyên lý hoạt động

- Mỗi bình luận trên YouTube được ánh xạ thành một vector duy nhất (document vector).
- Mỗi từ trong bình luận cũng được biểu diễn bởi một vector.
- Mô hình học song song các vector từ và vector văn bản, sau đó sử dụng để:
 - Dự đoán cảm xúc của bình luận dựa vào nội dung văn bản.
 - Tính toán sự tương đồng giữa các bình luận để hỗ trợ phân tích xu hướng.
- Khi có một bình luận mới, mô hình sẽ sinh ra vector biểu diễn cho bình luận đó và dự đoán nhãn cảm xúc tương ứng (Positive, Negative hoặc Neutral).

3.4. Cách triển khai Doc2Vec cho bài toán phân loại cảm xúc bình luận

3.4.1 Không sử dụng mô hình pretrain (huấn luyện từ đầu)

- Quy trình:
 - Làm sạch bình luận (xóa dấu, ký tự đặc biệt, bỏ URL, xử lý emoji, viết thường, ...).
 - Loại bỏ các bình luận rỗng hoặc không mang ý nghĩa.
 - Gắn nhãn (tagging) cho mỗi bình luận để mô hình biết văn bản nào thuộc về bình luận nào.
 - Huấn luyện Doc2Vec để sinh vector biểu diễn cho mỗi bình luận.
 - Sử dụng các vector biểu diễn đó để huấn luyện mô hình phân loại cảm xúc (như Logistic Regression, Naive Bayes, Decision Tree)

- Ưu điểm:
 - Vector biểu diễn phản ánh tốt nhất ngữ nghĩa của từng bình luận trong ngữ cảnh YouTube.
 - Có thể phát hiện được các mẫu ngôn ngữ đặc trưng trong bình luận (chẳng hạn như tiếng lóng, viết tắt, từ ngữ cảm xúc).
- Nhược điểm:
 - Cần một tập dữ liệu bình luận lớn để huấn luyện hiệu quả.
 - Tốn thời gian và tài nguyên tính toán để huấn luyện từ đầu.

3.4.2 Sử dụng mô hình *pretrain* (có sẵn)

- Quy trình:
 - Sử dụng mô hình Doc2Vec huấn luyện sẵn trên tập dữ liệu lớn (như Wikipedia, diễn đàn, tin tức).
 - Ánh xạ mỗi bình luận vào vector biểu diễn qua phương pháp `infer_vector()`.
 - Sử dụng các vector này để phân loại cảm xúc.
- Ưu điểm:
 - Tiết kiệm thời gian (không cần huấn luyện lại).
 - Vector đã được học từ nguồn dữ liệu lớn nên có khả năng khái quát tốt.
- Nhược điểm:
 - Không tối ưu hóa cho đặc thù ngôn ngữ trong bình luận YouTube (ví dụ: từ viết tắt, slang, meme, ngôn ngữ mạng xã hội).

3.5. Đánh giá mô hình Doc2Vec và những lưu ý khi huấn luyện

- Khác với các phương pháp truyền thống (BoW, TF-IDF), Doc2Vec tạo ra các vector có ý nghĩa về mặt ngữ nghĩa.

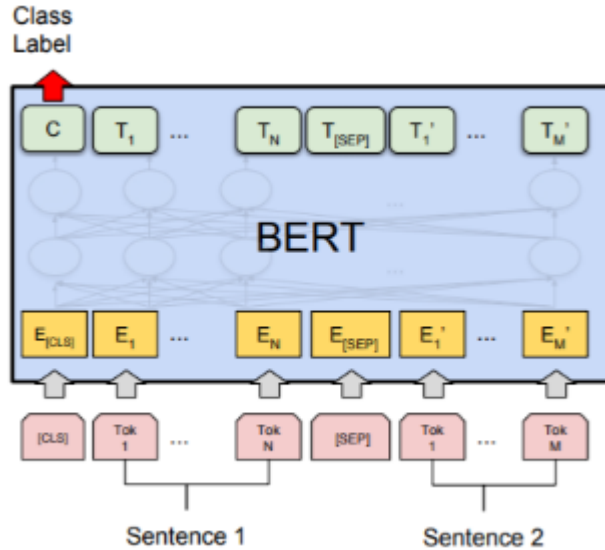
- Tuy nhiên, đánh giá hiệu quả vector Doc2Vec là khó khăn, vì không có chuẩn mực tuyệt đối (giống như việc đánh giá Word2Vec thông qua "king - man + woman = queen").
- Do đó, chất lượng vector thường được đánh giá gián tiếp qua độ chính xác của mô hình phân loại.
- Các tham số quan trọng cần lưu ý khi huấn luyện:
 - vector_size: số chiều của vector (ví dụ: 100, 200).
 - window: số từ xung quanh được sử dụng để học.
 - min_count: số lần xuất hiện tối thiểu của từ để được đưa vào từ điển.
 - epochs: số lần lặp lại toàn bộ dữ liệu.
- Do đó, chất lượng vector thường được đánh giá gián tiếp qua độ chính xác của mô hình phân loại.

3.6. Ví dụ minh họa

- Giả sử tập bình luận như sau:
 - "Video này rất hay, mình thích nội dung như thế này!" (Tích cực)
 - "Tệ quá, phí thời gian xem!" (Tiêu cực)
 - "Video cũng bình thường, không có gì đặc sắc lắm." (Trung lập)
- Quy trình:
 - Bước 1: Biểu diễn mỗi bình luận thành vector sử dụng Doc2Vec.
 - Bước 2: Dùng các vector này để huấn luyện mô hình phân loại cảm xúc thành 3 nhóm: Tích cực, Tiêu cực, Trung lập.
 - Bước 3: Khi có bình luận mới, mô hình sẽ sinh vector tương ứng và dự đoán cảm xúc của bình luận đó.

3.7. Giới thiệu về BERT

3.7.1. Mô hình BERT



Hình 3. 2 Mô hình Bert

- BERT (**Bidirectional Encoder Representations from Transformers**) là một mô hình học sâu dựa trên kiến trúc **Transformer**, giúp biểu diễn từ và câu theo cả hai chiều (**bidirectional**). Mô hình này được thiết kế để học ngữ nghĩa của từ trong câu dựa trên bối cảnh toàn cục, thay vì chỉ dựa vào từ liền kề như các mô hình trước đó.
- Không giống như Word2Vec hay Doc2Vec, vốn chỉ xét ngữ cảnh từ trái hoặc phải, BERT sử dụng cơ chế **self-attention** để xem xét toàn bộ câu cùng một lúc. Điều này giúp mô hình hiểu rõ hơn về mối quan hệ giữa các từ trong câu.

3.7.2. Các phiên bản mô hình BERT

- BERT có nhiều phiên bản khác nhau, phục vụ cho nhiều mục đích khác nhau:
 - **BERT Base & BERT Large**: Phiên bản gốc của Google với số lượng tham số khác nhau.
 - **PhoBERT**: Mô hình BERT được huấn luyện riêng cho tiếng Việt, giúp xử lý ngữ nghĩa tốt hơn so với BERT gốc.

- **DistilBERT**: Phiên bản nhẹ hơn của BERT, giảm số lượng tham số nhưng vẫn giữ hiệu suất cao.

3.7.3. Nguyên lý hoạt động của BERT

– BERT được huấn luyện theo hai nhiệm vụ chính:

- **Masked Language Model (MLM)**
 - **Cơ chế**: Khoảng 15% số từ trong câu sẽ bị thay thế bằng token [MASK]. Nhiệm vụ của mô hình là dự đoán từ bị che dựa vào bối cảnh xung quanh.
 - **Mục đích**: Giúp mô hình học cách hiểu nghĩa của từ dựa trên toàn bộ câu, thay vì chỉ dựa vào ngữ cảnh cục bộ.
- **Next Sentence Prediction (NSP)**
 - **Cơ chế**: Mô hình nhận đầu vào là một cặp câu. 50% số cặp này là các câu liên tiếp trong văn bản gốc (**IsNext**), 50% còn lại là các câu ngẫu nhiên không liên quan (**NotNext**).
 - **Mục đích**: Giúp mô hình hiểu quan hệ giữa các câu, từ đó hỗ trợ tốt hơn cho các tác vụ như tìm kiếm thông tin và trả lời câu hỏi.

3.8. Cách triển khai BERT cho bài toán phân loại cảm xúc bình luận

3.8.1. Không sử dụng mô hình pretrain (huấn luyện từ đầu)

- Quy trình:
 - Tiền xử lý dữ liệu: chuẩn hóa văn bản, loại bỏ ký tự đặc biệt, tách từ.
 - Tokenize dữ liệu bằng tokenizer của BERT.
 - Huấn luyện mô hình BERT trên tập dữ liệu bình luận.
 - Sử dụng vector đầu ra từ BERT để huấn luyện mô hình phân loại (Logistic Regression, Decision Tree, MLP).
- Ưu điểm:
 - Biểu diễn từ có ngữ cảnh chính xác hơn so với TF-IDF hoặc Doc2Vec.

- Có thể phát hiện được các mẫu bình luận phức tạp.
- Nhược điểm:
 - Cần nhiều dữ liệu và tài nguyên tính toán lớn để huấn luyện.

3.8.2. Sử dụng mô hình pretrain (PhoBERT)

- Quy trình:
 - Sử dụng mô hình BERT đã được huấn luyện trước trên tập dữ liệu lớn.
 - Tinh chỉnh (fine-tune) trên tập dữ liệu bình luận.
 - Sử dụng vector biểu diễn từ BERT để huấn luyện mô hình phân loại cảm xúc.
- Ưu điểm:
 - Tiết kiệm thời gian, không cần huấn luyện lại từ đầu.
 - Mô hình có thể tận dụng kiến thức từ dữ liệu lớn để cải thiện độ chính xác.
- Nhược điểm:
 - Có thể không tối ưu cho ngữ cảnh đặc thù của bình luận.

3.9. Hiện thực trên bài toán phân loại cảm xúc bình luận trên youtube

3.9.1. Mục tiêu của bài toán:

- Xây dựng mô hình phân loại cảm xúc bình luận trên YouTube dựa trên nội dung bình luận của người xem.
- Biểu diễn văn bản bằng các phương pháp khác nhau:
 - Bag of Words (CountVectorizer).
 - TF-IDF (TfidfVectorizer).
 - Doc2Vec (không pretrain và có sử dụng pretrain từ Word2Vec).
- Kết hợp các mô hình học máy như Logistic Regression và Naive Bayes để thực hiện phân loại.
- Dữ liệu được thu thập từ YouTube và lưu vào file comments.csv, gồm 2 cột:
 - comment_text: Nội dung bình luận.

- sentiment: Nhận cảm xúc (tích cực, tiêu cực, trung lập).

3.9.2. Nội dung các bước thực hiện chi tiết

a. Mô hình không sử dụng pretrain

- Để chuẩn bị dữ liệu cho mô hình, nhóm thực hiện một chuỗi các bước **tiền xử lý** nhằm làm sạch và chuẩn hóa văn bản tương tự như ở phần 2.8.2 (Áp dụng phương pháp học máy cho bài toán phân loại cảm xúc).
- Nhóm sử dụng Doc2Vec, để chuyển các bình luận thành vector đặc trưng. Đầu tiên, dữ liệu được chia thành 70% huấn luyện và 30% kiểm tra, sau đó **chuyển thành định dạng TaggedDocument** để phù hợp với Doc2Vec.

```
train_tagged = train.apply(
    lambda r:
    TaggedDocument(words=tokenize_text(r['clean_comment']),
    tags=[str(r.Sentiment)]), axis=1)

test_tagged = test.apply(
    lambda r:
    TaggedDocument(words=tokenize_text(r['clean_comment']),
    tags=[str(r.Sentiment)]), axis=1)
```

- Sau chuyển đổi các tài liệu đã gán nhãn (TaggedDocument) thành vector đặc trưng và nhãn tương ứng để sử dụng trong việc huấn luyện hoặc kiểm tra mô hình phân loại. Các vector từ Doc2Vec được sử dụng làm đầu vào cho mô hình Logistic Regression.

```
def vec_for_learning(model, tagged_docs):
    sents = tagged_docs.values
    targets, regressors = zip(
        *[ (doc.tags[0], model.infer_vector(doc.words,
        epochs=20)) for doc in sents]
    )
    return targets, regressors
```

- Mô hình Doc2Vec được sử dụng trong trường hợp này là Distributed Memory Mean (DMM). Đây là một biến thể của Doc2Vec, tập trung vào việc dự đoán từ dựa trên ngữ cảnh và nhãn của tài liệu, với cách kết hợp vector từ và vector tài liệu bằng phép tính trung bình, tham số bao gồm:
 - `dm=1`: Chỉ định sử dụng mô hình Distributed Memory (DM). Giá trị `dm=1` xác nhận đây là phiên bản DMM.
 - `dm_mean=1`: Xác định cách kết hợp vector từ và vector tài liệu bằng cách lấy trung bình (mean), thay vì nối (concatenate).
 - `vector_size`: Kích thước chiều của vector đặc trưng.
 - `window`: Kích thước cửa sổ ngữ cảnh, cho phép mô hình học mối quan hệ giữa các từ trong một phạm vi rộng.
 - `negative`: Sử dụng mẫu tiêu cực trong quá trình huấn luyện để tăng khả năng phân biệt của mô hình.
 - `min_count`: Giữ lại các từ xuất hiện ít nhất `min_count` lần trong tập dữ liệu.
 - `workers`: Số lượng luồng xử lý song song.
 - `alpha` và `min_alpha`: Tốc độ học ban đầu và tốc độ học tối
- Mô hình được huấn luyện qua **30 epochs**, với tốc độ học (learning rate) giảm dần từ **0.065 xuống 0.001** nhằm tối ưu hóa quá trình học

```
model_dmm = Doc2Vec(dm=1, dm_mean=1, vector_size=300,
                    window=10, negative=5, min_count=1, workers=5,
                    alpha=0.065, min_alpha=0.001)

initial_alpha = 0.065
min_alpha = 0.001
alpha_decay = (initial_alpha - min_alpha) / 30
```

```

for epoch in range(30):
    model_dmm.train(utils.shuffle([x for x in
tqdm(train_tagged.values)]),
total_examples=len(train_tagged.values), epochs=1)
    model_dmm.alpha = max(model_dmm.alpha - alpha_decay,
min_alpha)
    model_dmm.min_alpha = model_dmm.alpha

```

– **Kết quả huấn luyện và đánh giá:**

```

y_train, X_train = vec_for_learning(model_dmm,
train_tagged)
y_test, X_test = vec_for_learning(model_dmm, test_tagged)

logreg.fit(X_train, y_train)
y_pred = logreg.predict(X_test)
print('Testing accuracy %s' % accuracy_score(y_test,
y_pred))
print('Testing F1 score: {}'.format(f1_score(y_test,
y_pred, average='weighted'))))

```

- **Đánh giá về độ chính xác của mô hình:**

```

Testing accuracy 0.6255550074000986
Testing F1 score: 0.6239432453928653

```

- **Thử nghiệm mô hình:**

```

def predict_new_text(new_text, model_dmm, classifier):
    words = new_text.split()
    vec_dmm = model_dmm.infer_vector(words, epochs=20)

    # In ra vector đặc trưng của văn bản
    print("Vector representation:", vec_dmm)

    return classifier.predict([vec_dmm])[0]

new_text = "ai chê chứ tui thấy hay mà"
predicted_label = predict_new_text(new_text, model_dmm,
logreg)
print("Predicted Product:", predicted_label)

```

▪ Kết quả thử nghiệm:

Vector representation: [-7.90863670e-03 4.53630602e-03
3.86206689e-03 -6.73785456e-04
-3.65928677e-03 6.89599616e-03 -4.10180510e-04 -6.62320619e-03
3.97242955e-04 3.51617287e-04 6.46437658e-03 -1.35176131e-04
9.12111346e-03 6.36902289e-04 -4.83221654e-03 -7.52357487e-03
4.91763465e-04 -3.45488335e-03 2.84196204e-03 1.07101619e-03
2.03433353e-03 -8.74241255e-03 8.86199716e-03 2.51833140e-03
2.84792599e-03 -2.24609324e-03 1.92524272e-03 9.77985468e-03
-5.61459269e-03 -8.54982622e-03 -3.15692718e-03 -1.73051318e-03
-5.54104429e-03 -5.34021622e-03 1.73979509e-03 -5.40808355e-03
-1.10980170e-03 -4.74695023e-03 -5.06545231e-03 5.52960066e-03
4.34644846e-03 1.34598082e-02 -9.16335359e-03 -8.19776393e-03
-1.01147490e-02 -9.90287308e-03 2.18259147e-03 4.44535166e-03
-3.38344299e-03 7.51478644e-03 -3.62931332e-03 -1.91776233e-03
-5.62601315e-04 -7.12470012e-03 6.01223763e-03 -5.03432611e-03
5.19718800e-04 4.76849126e-03 -9.94074438e-03 3.89259262e-03
1.28007401e-03 8.64859018e-03 -1.24942220e-03 3.24360863e-03
-5.91624303e-05 -1.70140038e-03 7.71332718e-03 6.95910247e-04
-8.85696802e-03 4.85304801e-04 -6.74210198e-04 -4.34888154e-03
1.37441303e-03 2.68957927e-03 2.74498644e-03 6.14207378e-03
5.09325555e-03 -2.96896789e-03 -1.88592868e-03 1.12466542e-02
-8.16757884e-03 -1.65867282e-03 9.56241682e-04 5.59217995e-03
-7.35567184e-03 4.82913246e-03 1.34991563e-03 -8.31957208e-04
-1.03530223e-02 -3.50364065e-03 -4.46555577e-03 7.47341488e-04
-2.90172081e-03 -8.21452308e-03 -4.81909810e-05 -1.73989008e-03
2.83015217e-03 1.69105362e-03 -9.63518489e-03 1.38355768e-03
-1.24236336e-03 -4.75682085e-03 -1.03627564e-02 3.32938251e-03
1.96276442e-03 -3.63856764e-03 -1.72062148e-03 7.98212888e-04
-1.63436972e-03 -1.18009993e-04 2.33521569e-03 -9.75741248e-04
-8.45436472e-03 1.03637797e-03 8.65044072e-04 6.78257737e-03
-1.08581269e-02 7.39860348e-03 8.33517592e-03 3.65561224e-04
-1.25918398e-03 -2.16212677e-04 4.36349865e-03 -8.35033227e-03
2.85841292e-03 3.38503742e-03 -3.13315075e-03 1.02009270e-02
7.84663949e-04 -5.00570657e-03 2.21181996e-02 1.31473560e-02
-4.01474629e-03 1.92239985e-03 1.27950227e-02 1.57562213e-03
-2.37423903e-03 2.83188332e-04 4.53568390e-03 -9.51963768e-04
3.51029565e-03 -5.70059661e-03 -9.11968760e-03 -7.73175294e-03
-1.63599930e-03 1.65567372e-03 5.75209968e-03 3.76574602e-03
1.72952830e-04 5.13611967e-03 1.22400578e-02 -4.68677748e-03
2.29670736e-03 9.61978082e-03 -5.76750049e-03 -2.93901889e-03
-3.06467968e-03 -4.58204281e-03 3.60470294e-04 -3.52228736e-03
7.25384662e-03 -5.65226795e-03 -1.79067790e-03 8.80885031e-03
1.07830781e-02 -6.13005785e-03 1.21202143e-02 8.92546121e-03
1.58016123e-02 4.91274288e-03 -7.43530085e-03 9.72593611e-04
-1.04606152e-02 -1.15392264e-03 -5.57908276e-03 3.71562107e-03
-1.16039162e-04 4.38571349e-03 7.64171220e-03 -1.24404551e-02
-3.61719052e-03 7.57264392e-03 6.37629628e-03 -1.05199190e-02
6.03411288e-04 1.53493159e-03 9.19387490e-03 4.91403788e-03
6.52466994e-03 8.18016380e-03 -2.09984696e-03 7.05492543e-03

```

-4.63477476e-03 -9.17492528e-03  4.49507032e-03 -6.32995361e-05
 6.70160213e-03  3.22130800e-04  4.66935366e-04  7.21329171e-03
-7.57260481e-03 -1.27303023e-02  8.82893743e-04 -6.40464947e-03
-5.89214498e-03  1.00137554e-02 -9.79185686e-04  4.15885122e-03
-1.40560707e-02 -1.49435899e-03  2.95258197e-03  6.31336216e-03
 5.77249052e-03 -6.61472883e-03 -1.03659444e-02  7.22608715e-03
 3.59158008e-03 -2.64806743e-03  7.98201654e-03  8.04602460e-04
 5.5579923e-03 -7.34820438e-04 -8.09872057e-03 -9.46772750e-04
 2.09960178e-03  1.70446379e-04 -7.07838032e-03  2.98662786e-03
-8.01921077e-03 -2.64740176e-03  5.07190882e-04  2.60997401e-03
-8.07553809e-03 -4.22560464e-04 -5.89977717e-04  5.76719316e-03
-3.40519776e-03 -3.29395779e-03 -1.92092918e-03  3.49521800e-03
 4.91273357e-03 -9.81706753e-03 -8.42487067e-03 -6.76045613e-03
 5.11378888e-03  4.14468115e-04  6.91651460e-03  1.81147270e-03
-1.17937592e-03  7.44784856e-03 -4.27404884e-03 -1.16027780e-02
-6.74806349e-03 -3.46755376e-03 -7.58570805e-03 -1.42524962e-03
 1.23490428e-03  5.62772853e-03  3.68974288e-03 -4.50465130e-03
 1.13080805e-02 -1.66784867e-03 -8.15231586e-04  7.12088263e-03
 1.85483822e-03  4.33926424e-03  6.50889473e-03  4.34547081e-04
-1.26917753e-03 -2.56481185e-03 -6.86417799e-03 -1.22360373e-03
-2.32010987e-03 -2.61441281e-04  5.33106178e-03  3.17012938e-03
-5.30382432e-03  1.57204410e-03 -7.41699012e-04  1.72862783e-03
 7.63300760e-03 -2.03839829e-03  8.55332892e-03  9.17907339e-03
-2.67211604e-03 -1.22604484e-04 -1.72113057e-03 -3.47065739e-03
-4.54085879e-03  8.29700939e-03 -1.07675632e-02  8.76493193e-03
 2.41624191e-03  2.98027718e-03  9.58613679e-03 -2.42072600e-03
-6.78226491e-03  4.52087447e-03  5.98564465e-03  2.99105770e-04]

```

Predicted Product: Pos

- **Nhận xét mô hình:**

- Độ chính xác (Accuracy): Mô hình đạt độ chính xác khoảng 62.56%. Đây là một kết quả trung bình, không quá cao, cho thấy mô hình có khả năng phân loại đúng ở mức chấp nhận được nhưng chưa thực sự tốt.
- F1 Score: F1 Score (trung bình có trọng số) là 62.39%, rất gần với độ chính xác. F1 Score là trung bình điều hòa giữa Precision (độ chính xác) và Recall (độ bao phủ). Giá trị F1 Score này cũng ở mức trung bình, tương tự như độ chính xác, cho thấy mô hình không có sự chênh lệch lớn giữa Precision và Recall, nhưng hiệu suất tổng thể vẫn chưa cao.
- So sánh Accuracy và F1 Score: Hai chỉ số này khá gần nhau, điều này cho thấy dữ liệu có thể không bị mất cân bằng nghiêm trọng (class imbalance). Nếu dữ liệu bị mất cân bằng (ví dụ: một lớp chiếm ưu thế hơn các lớp khác), F1 Score thường sẽ thấp hơn Accuracy đáng kể. Ở đây, sự chênh lệch nhỏ giữa hai chỉ số cho thấy mô hình hoạt động tương đối ổn định trên các lớp.

- **Đánh giá tổng quan:** Với độ chính xác và F1 Score ở mức ~62%, mô hình Doc2Vec kết hợp Logistic Regression có hiệu suất trung bình. Hiệu suất này cần phải được cải thiện.
- **Dự đoán nhãn:** Mô hình dự đoán nhãn Pos (tích cực), điều này phù hợp với ý nghĩa của câu "ai chê chứ tui thấy hay mà", vì câu này thể hiện một cảm xúc tích cực ("tui thấy hay mà"). Kết quả dự đoán này là chính xác trong trường hợp này, cho thấy mô hình có khả năng nhận diện cảm xúc tích cực trong một số trường hợp.

b. Mô hình sử dụng pretrain(BERT)

– Import các thư viện cần thiết

```
from transformers import AutoTokenizer,
AutoModelForSequenceClassification
import torch
```

- transformers: Thư viện của Hugging Face giúp tải và sử dụng các mô hình NLP pretrained.
- torch: Thư viện machine learning hỗ trợ deep learning, giúp thực hiện tính toán tensor trên GPU/CPU.
- AutoTokenizer: Dùng để chuyển đổi văn bản thành tensor để mô hình hiểu được.
- AutoModelForSequenceClassification: Dùng để tải mô hình pretrained phục vụ phân loại câu theo cảm xúc.

– Khởi tạo mô hình phân loại cảm xúc sử dụng BERT

```
# Use a pipeline as a high-level helper
import torch
from transformers import pipeline

pipe = pipeline("text-classification",
model="UchihaMadara/phobert-finetuned-sentiment-
analysis")

# Load model directly
from transformers import AutoTokenizer,
AutoModelForSequenceClassification
```

```
tokenizer =
AutoTokenizer.from_pretrained("UchihaMadara/phobert-
finetuned-sentiment-analysis")
model =
AutoModelForSequenceClassification.from_pretrained("Uchi
haMadara/phobert-finetuned-sentiment-analysis")
```

- `import torch`: Nhập thư viện PyTorch, một thư viện học sâu phổ biến, được sử dụng để xử lý tensor và thực hiện các phép tính trên mô hình học sâu.
- `from transformers import pipeline`: Nhập hàm `pipeline` từ thư viện `transformers` của Hugging Face. Hàm `pipeline` là một công cụ cấp cao (high-level helper) giúp đơn giản hóa việc sử dụng các mô hình đã được huấn luyện sẵn cho các tác vụ như phân loại văn bản, dịch thuật, v.v.
- `pipe = pipeline("text-classification", ...)`: Khởi tạo một đối tượng `pipeline` cho tác vụ phân loại văn bản (text-classification). Hàm `pipeline` sẽ tự động tải mô hình và tokenizer tương ứng để thực hiện tác vụ này.
- `model="UchihaMadara/phobert-finetuned-sentiment-analysis"`: Chỉ định tên mô hình được sử dụng, ở đây là `UchihaMadara/phobert-finetuned-sentiment-analysis`. Đây là một mô hình PhoBERT (một biến thể của BERT được tối ưu cho tiếng Việt) đã được tinh chỉnh (fine-tuned) trên dữ liệu tiếng Việt để phân loại cảm xúc thành các nhãn như `negative`, `neutral`, hoặc `positive`.
- `model = AutoModelForSequenceClassification.from_pretrained("UchihaMadara/phobert-finetuned-sentiment-analysis")`: Tải mô hình phân loại chuỗi đã được huấn luyện sẵn với tên `UchihaMadara/phobert-finetuned-sentiment-analysis`. Mô hình này là một phiên bản PhoBERT đã được tinh chỉnh để phân loại cảm xúc văn bản tiếng Việt thành các nhãn `negative`, `neutral`, hoặc `positive`.

- Chi tiết về mô hình: `AutoModelForSequenceClassification` là một lớp chuyên dụng cho các bài toán phân loại chuỗi (sequence classification), chẳng hạn như phân loại cảm xúc (sentiment analysis). Mô hình này bao gồm một tầng đầu ra (classification head) được thêm vào trên PhoBERT để dự đoán nhãn.

– **Đưa mô hình về chế độ đánh giá (evaluation mode)**

```
model.eval() # Đưa về chế độ đánh giá
```

- `model.eval()`: Đưa mô hình vào chế độ đánh giá (evaluation mode). Trong chế độ này, các tầng như Dropout hoặc Batch Normalization (nếu có) sẽ hoạt động ở trạng thái suy luận (inference), nghĩa là không áp dụng các kỹ thuật regularization như Dropout (Dropout sẽ bị tắt).

– **Xác định danh sách nhãn cho mô hình**

```
# Danh sách nhãn của mô hình
LABELS = ["negative", "neutral", "positive"]
```

- Mô hình này có 3 lớp đầu ra tương ứng với cảm xúc: tiêu cực (negative), trung lập (neutral) và tích cực (positive).

– **Hàm phân loại cảm xúc của văn bản**

```
def classify_text(text):
    """
    Phân loại comment thành positive, neutral hoặc
    negative.
    """
    # Tokenize input
    inputs = tokenizer(text, return_tensors="pt",
padding=True, truncation=True, max_length=512)

    # Dự đoán
    with torch.no_grad():
        outputs = model(**inputs)
        print(outputs.logits.softmax(dim=-1).tolist())

    # Lấy giá trị logits
    logits = outputs.logits
```

```
# Chuyển thành nhãn dự đoán
predicted_class = torch.argmax(logits, dim=1).item()

return LABELS[predicted_class]
```

- `def classify_text(text)`: Định nghĩa hàm `classify_text` để phân loại cảm xúc của một đoạn văn bản (`text`) thành `positive`, `neutral`, hoặc `negative`.
- `inputs = tokenizer(text, return_tensors="pt", padding=True, truncation=True, max_length=512)`: Tiền xử lý văn bản đầu vào bằng `tokenizer`:
- `text`: Văn bản cần phân loại.
- `return_tensors="pt"`: Yêu cầu `tokenizer` trả về kết quả dưới dạng tensor của PyTorch ("`pt`" là viết tắt của PyTorch).
- `padding=True`: Tự động thêm padding (đệm) để đảm bảo độ dài đầu vào phù hợp với yêu cầu của mô hình.
- `truncation=True`: Cắt ngắn văn bản nếu nó vượt quá độ dài tối đa (`max_length`).
- `max_length=512`: Độ dài tối đa của chuỗi đầu vào là 512 token (đây là giới hạn thông thường của BERT và các biến thể như PhoBERT).

– Kiểm thử mô hình với một số câu

```
text_1 = "tui thấy bài này cũng cũng"
text_2 = "bài này mà cũng ra mắt được"
text_3 = "ai chê chứ tui thấy cũng hay mà"

print(f"Text: {text_1} → Class: {classify_text(text_1)}")
print(f"Text: {text_2} → Class: {classify_text(text_2)}")
print(f"Text: {text_3} → Class: {classify_text(text_3)}")
```

[[0.0031538973562419415, 0.9934818148612976, 0.0033643413335084915]]

Text: tui thấy bài này cũng cũng → Class: neutral

[[0.07108806818723679, 0.6250019669532776, 0.30390989780426025]]

Text: bài này mà cũng ra mắt được → Class: neutral

[[0.0027305593248456717, 0.005289058201014996, 0.9919804334640503]]

Text: ai chê chứ tui thấy cũng hay mà → Class: positive

- Text 1: "tui thấy bài này cũng cũng"
 - Xác suất: [0.0031538973562419415, 0.9934818148612976, 0.0033643413335084915]
 - negative: 0.00315 (~0.32%)
 - neutral: 0.99348 (~99.35%)
 - positive: 0.00336 (~0.34%)
 - Dự đoán: neutral
- Text 2: "bài này mà cũng ra mắt được"
 - Xác suất: [0.07108806818723679, 0.6250019669532776, 0.30390989780426025]
 - negative: 0.07109 (~7.11%)
 - neutral: 0.62500 (~62.50%)
 - positive: 0.30391 (~30.39%)
 - Dự đoán: neutral
- Text 3: "ai chê chứ tui thấy cũng hay mà"
 - Xác suất: [0.0027305593248456717, 0.005289058201014996, 0.9919804334640503]
 - negative: 0.00273 (~0.27%)
 - neutral: 0.00529 (~0.53%)
 - positive: 0.99198 (~99.20%)
 - Dự đoán: positive
- **Nhận xét:**

▪ **Độ chính xác:**

- Mô hình dự đoán đúng 2/3 trường hợp (Text 1 và Text 3), đạt tỷ lệ chính xác khoảng 66.67% trên tập thử nghiệm nhỏ này.
- Trường hợp sai (Text 2) cho thấy mô hình có thể gặp khó khăn trong việc nhận diện các câu mang sắc thái tiêu cực nhẹ hoặc có ngữ nghĩa phức tạp (như sự mỉa mai, nghi ngờ).

▪ **Độ tự tin (Confidence):**

- Trong các trường hợp dự đoán đúng (Text 1 và Text 3), mô hình có độ tự tin rất cao (xác suất $> 99\%$), cho thấy mô hình hoạt động tốt khi cảm xúc được thể hiện rõ ràng (rõ ràng trung tính hoặc rõ ràng tích cực).
- Trong trường hợp dự đoán sai (Text 2), xác suất cho nhãn neutral chỉ là 62.50%, và xác suất cho positive (30.39%) và negative (7.11%) không quá thấp, cho thấy mô hình không hoàn toàn tự tin với dự đoán. Điều này có thể là dấu hiệu của việc mô hình chưa được huấn luyện đủ tốt trên các câu mang sắc thái tiêu cực nhẹ hoặc mỉa mai.

3.10. So sánh giữa Doc2Vec và BERT (from scratch)

Tiêu chí	Tự huấn luyện Doc2Vec	Sử dụng BERT pretrain
Độ phù hợp với bình luận	Cao, vì học đúng nội dung cần phân loại	Rất cao, vì học từ ngữ cảnh và bối cảnh 2 chiều
Thời gian	Lâu, tốn tài nguyên	Rất lâu, cần GPU

huấn luyện		mạnh
Yêu cầu về dữ liệu	Cần dữ liệu lớn và đủ mẫu	Cần lượng dữ liệu lớn để fine-tune tốt
Khả năng khái quát	Có thể tốt nếu dữ liệu đủ đa dạng	Xuất sắc, vì đã huấn luyện trên dữ liệu rất lớn
Khả năng mở rộng	Có thể khó nếu dữ liệu tăng nhiều	Dễ mở rộng với fine-tuning, nhưng cần tài nguyên mạnh

Bảng 3. 1 So sánh giữa Doc2Vec và BERT

CHƯƠNG 4 – ĐÁNH GIÁ VÀ PHÂN TÍCH KẾT QUẢ

4.1. So sánh các phương pháp biểu diễn văn bản

Tiêu chí	Bag of Words (BoW)	TF-IDF	Doc2Vec	BERT
Cách biểu diễn	Vector tần suất xuất hiện từ	Vector tần suất từ điều chỉnh theo mức độ quan trọng	Vector ngữ nghĩa liên tục biểu diễn toàn văn bản	Vector ngữ nghĩa theo từng từ với ngữ cảnh
Giữ thông tin ngữ cảnh	Không	Không	Có (thông qua học ngữ nghĩa và bối cảnh từ)	Rất cao (học bối cảnh hai chiều)
Độ phức tạp	Đơn giản, dễ triển khai	Trung bình, yêu cầu thêm bước tính IDF	Cao, yêu cầu mô hình học sâu và huấn luyện phức tạp	Rất cao, yêu cầu Transformer
Kích thước vector	Rất lớn, phụ thuộc số từ trong từ điển	Rất lớn, phụ thuộc số từ trong từ điển	Nhỏ gọn, cố định (do chọn trước vector_size)	Lớn, phụ thuộc số token trong câu
Khả năng biểu diễn ngữ nghĩa sâu	Kém	Kém	Tốt (nhắm được mối quan hệ từ, bối cảnh văn bản)	Xuất sắc (hiểu ngữ cảnh hai chiều)

Khả năng phân biệt văn bản ngắn, dài	Trung bình, dễ bị nhiễu với văn bản dài	Khá tốt với từ quan trọng	Rất tốt, vì giữ ngữ nghĩa tổng thể của toàn văn bản	Rất tốt, xử lý từng từ với bối cảnh
---	---	---------------------------	---	-------------------------------------

Bảng 4. 1 So sánh các phương pháp biểu diễn văn bản

Tiêu chí	CountVector	Binary Vector	TF-IDF Vector	Hashing Vector	N-gram
Cách biểu diễn	Đếm số lần từ xuất hiện	Đánh dấu có/không	Cân bằng giữa tần suất & phổ biến	Băm từ thành chỉ số cố định	Kết hợp nhiều từ liên tiếp
Kích thước vector	Cao	Cao nhưng đơn giản	Cao nhưng phân biệt tốt hơn	Cố định, không lưu vocab	Tăng nhanh theo độ dài n-gram
Độ hiểu quả	Trung bình	Khá tốt với Naive Bayes	Tốt nhất với Logistic	Khá ổn, hiệu quả với Random Forest	Tăng nhẹ độ chính xác nhưng tốn RAM

❖ So sánh về tính đầy đủ và chính xác của dữ liệu

Tiêu chí	Tập dữ liệu BoW/TF-IDF	Tập dữ liệu Doc2Vec	Tập dữ liệu BERT
-----------------	-------------------------------	----------------------------	-------------------------

Độ đầy đủ thông tin	Thấp: Mất thông tin ngữ cảnh, cấu trúc văn bản	Cao: Giữ được ý nghĩa tổng thể văn bản	Rất cao: Hiểu ngữ cảnh hai chiều
Xử lý từ đồng nghĩa, đa nghĩa	Không xử lý	Có khả năng xử lý thông qua học biểu diễn liên tục	Xuất sắc, hiểu ngữ cảnh từng từ
Khả năng áp dụng với dữ liệu nhỏ	Tốt với dữ liệu nhỏ (nếu từ khóa rõ ràng)	Khó (cần dữ liệu nhiều để học biểu diễn tốt)	Có thể áp dụng tốt với fine-tuning
Khả năng tổng quát hóa	Kém, dễ overfit với dữ liệu nhỏ hoặc đặc biệt	Tốt, biểu diễn học từ ngữ nghĩa tổng thể	Rất tốt, do đã huấn luyện trên tập dữ liệu lớn
Khả năng mở rộng, thêm dữ liệu mới	Cần huấn luyện lại toàn bộ	Có thể sử dụng mô hình pretrain để tiết kiệm công sức	Dễ dàng fine-tune trên dữ liệu mới

Bảng 4. 2 So sánh về tính đầy đủ và chính xác của dữ liệu

❖ So sánh các phương pháp học máy đã áp dụng

Tiêu chí	Naive Bayes	Logistic Regression	Decision Tree	Random Forest	KNN
Cơ chế hoạt động	Dựa trên xác suất có điều kiện, giả định độc	Phân loại tuyến tính, tối ưu theo xác suất	Phân tách dựa vào điều kiện đặc trưng (cây quyết định)	Tập hợp nhiều cây quyết định, bỏ phiếu để phân loại	Dựa vào khoảng cách giữa điểm dữ liệu và hàng xóm gần nhất

	lập từ				
Độ chính xác với BoW/TF-IDF	Trung bình (tốt khi từ khóa rõ, độc lập)	Cao (xử lý tốt dữ liệu tuyến tính)	Trung bình (phụ thuộc cách phân nhánh)	Cao nhất trong các mô hình đã thử nghiệm	Khá cao, nhưng nhạy cảm với nhiễu và dữ liệu phân tán
Khả năng học dữ liệu phức tạp	Kém (vì giả định độc lập từ)	Khá tốt, nhưng khó với dữ liệu phi tuyến	Trung bình, dễ overfit	Tốt, kháng overfitting nhờ mô hình tổ hợp	Kém với dữ liệu nhiều chiều hoặc không phân tách rõ
Tốc độ huấn luyện	Rất nhanh, đơn giản	Nhanh	Trung bình (chậm hơn NB, LR)	Chậm hơn do huấn luyện nhiều cây	Chậm với tập dữ liệu lớn (tính khoảng cách cho nhiều điểm)
Giải thích mô hình (interpretability)	Dễ giải thích (công thức xác suất)	Khá dễ giải thích qua trọng số	Dễ hiểu (cây quyết định trực quan)	Khó giải thích do tính tổ hợp	Khó giải thích (dựa vào khoảng cách)
Khả năng mở rộng	Khó mở rộng với đặc trưng phức tạp	Tốt, dễ mở rộng	Cần kiểm soát độ sâu để tránh overfitting	Tốt, ổn định với dữ liệu lớn nếu điều chỉnh tốt	Kém, không hiệu quả với dữ liệu lớn hoặc nhiễu

Bảng 4. 3 So sánh các phương pháp học máy đã áp dụng

4.2. Tổng kết và lựa chọn giải pháp hiệu quả

- Biểu diễn BoW và TF-IDF:
 - Ưu điểm: Đơn giản, dễ triển khai, hiệu quả khi các từ khóa thể hiện rõ ràng cảm xúc (ví dụ: "tuyệt vời" cho tích cực, "tệ" cho tiêu cực).
 - Nhược điểm: Mất ngữ cảnh, không thể xử lý tốt các bình luận phức tạp, chứa nhiều sắc thái cảm xúc hoặc mang ý nghĩa mỉa mai.
- Biểu diễn Doc2Vec:
 - Ưu điểm: Biểu diễn sâu, giữ được ý nghĩa văn bản, phù hợp cho bài toán nhiều dạng ngôn ngữ (mô tả công việc phức tạp, chi tiết).
 - Nhược điểm: Cần dữ liệu lớn, tốn nhiều thời gian và tài nguyên để huấn luyện tốt.
- Biểu diễn BERT:
 - Ưu điểm: Hiểu ngữ cảnh tốt nhất, xử lý văn bản dài và phức tạp, thích hợp khi cần độ chính xác cao.
 - Nhược điểm: Yêu cầu tài nguyên lớn (GPU), thời gian huấn luyện lâu.
- Mô hình học máy truyền thống:
 - Naive Bayes: Đơn giản, nhanh nhưng kém hiệu quả với văn bản phức tạp.
 - Logistic Regression: Cân bằng giữa tốc độ và độ chính xác, tốt nhất khi kết hợp với TF-IDF.
 - Decision Tree: Dễ hiểu nhưng dễ overfit, kém hiệu quả với vector thưa như BoW.
 - Random Forest khắc phục điểm yếu của cây đơn lẻ, đạt độ chính xác cao nhất trong demo.
 - KNN cho kết quả khá nhưng chậm và nhạy cảm với nhiễu.

4.3. Kết luận lựa chọn giải pháp phù hợp cho đề tài

Tiêu chí	Giải pháp hiệu quả nhất
Biểu diễn văn bản	TF-IDF (dữ liệu nhỏ), Doc2Vec (dữ liệu lớn), BERT (yêu cầu độ chính xác cao, dữ liệu lớn, tài nguyên mạnh)
Phương pháp học máy	Logistic Regression (cân bằng tốc độ và hiệu quả)
Dữ liệu và mở rộng	TF-IDF + Logistic Regression cho giai đoạn đầu, sử dụng BERT hoặc PhoBERT nếu mở rộng dữ liệu

Bảng 4. 4 Lựa chọn giải pháp phù hợp cho đề tài

4.4. Đề xuất giải pháp và hướng phát triển cho bài toán "Phân loại cảm xúc bình luận trên youtube"

4.4.1. Giai đoạn 1 (Dữ liệu ít, triển khai nhanh):

TF-IDF + Logistic Regression để đạt hiệu quả nhanh, độ chính xác cao.

4.4.2. Giai đoạn 2 (Dữ liệu mở rộng, ngành nghề đa dạng):

Doc2Vec (huấn luyện hoặc pretrain) + MLP (Deep Learning) để tăng độ chính xác, xử lý tốt các bình luận dài, phức tạp.

BERT hoặc PhoBERT nếu yêu cầu độ chính xác cao.

4.4.3. Hướng phát triển:

Kết hợp thêm BERT hoặc transformer models để biểu diễn ngữ nghĩa nâng cao hơn khi dữ liệu lớn hơn và yêu cầu cao hơn.

Fine-tune PhoBERT trên tập dữ liệu bình luận để tối ưu hóa cho bài toán cụ thể

TÀI LIỆU THAM KHẢO

[1] Giới thiệu về Mô hình Bag-of-Words (BoW)

Access: https://pyimagesearch-com.translate.goog/2022/07/04/introduction-to-the-bag-of-words-bow-model/? x tr sl=en& x tr tl=vi& x tr hl=vi& x tr_pto=tc

[2] Naive Bayes Classifiers

Access: <https://www.geeksforgeeks.org/naive-bayes-classifiers/>

[3] Áp dụng thuật toán Naive Bayes giải quyết bài toán phân loại email độc hại

Access: <https://www.studocu.com/de-at/document/truong-dai-hoc-duy-tan/computer-science/ap-dung-thuat-toan-naive-bayes-giai-quyet-bai-toan-phan-loai-email-doc-hai/114309967?origin=search-results>

[4] Logistic Regression in Machine Learning

Access: <https://www.geeksforgeeks.org/understanding-logistic-regression/>

[5] Decision Tree algorithm

Access: <https://www.geeksforgeeks.org/naive-bayes-classifiers/>

[6] K-Nearest Neighbor(KNN) Algorithm

Access: <https://www.geeksforgeeks.org/k-nearest-neighbours/>

[7] Random Forest algorithm

Access: https://machinelearningcoban.com/tabml_book/ch_model/random_forest.html

[8] Doc2Vec in NLP

Access: <https://www.geeksforgeeks.org/doc2vec-in-nlp/>

[9] TF-IDF trong xử lý ngôn ngữ tự nhiên

Access: <https://viblo.asia/p/doi-net-ve-tf-idf-trong-xu-ly-ngon-ngu-tu-nhien-vyDZO0e9lwj>

[10] Chat GPT